

Midterm Project

12211175 손승우

1. 학습 목적

이 프로젝트의 학습 목적은, Affine, Homography 및 DLT 모델을 통한 기초 변환 매트릭스들을 생성해본다. 결과 분석을 통해 각 변환에 있어서의 세부적인 특징들과 주의점을 알아보고, 최종적으로는 정밀센서모델과 보정된 EOP를 활용한 카메라 모델링을 통해 실세계 좌표계와 영상좌표계 간의 정확한 변환 체계를 구축하고, 이를 실험적으로 검증하는 전 과정을 체득하는 데 있다.

2. 배경이론

- **동차 좌표계 (Homogeneous Coordinates)** : 투영기하학에서 사용되는 좌표계이다. 기하 변환을 행렬곱으로 표현하기 위한 좌표계로, 원래 좌표에 차원을 하나 추가하여 표현하며, 투영 관계를 표현하기에 적합하다. 유클리드 좌표와는 아래와 같은 관계를 가지고 있다.

$$[x, y, w] \xrightarrow{\text{변환}} \left(\frac{x}{w}, \frac{y}{w} \right)$$

- **Affine Transformation(아핀 변환)** : 아핀 변환은 2차원 평면상의 점에 대한 선형변환과 평행이동을 포함하는 일반적인 2D 기하학적 변환이다. 회전, 확대/축소, 전단(shear), 이동을 모두 표현할 수 있는 변환이다. 동차좌표계를 활용하여 아래와 같은 변환 matrix를 구성할 수 있다.

6개 자유도: translation(2), rotation(1), Scale(2), Shear(1)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **Homography Transformation(투영 변환, 호모그래피)** : 호모그래피는 하나의 평면이 다른 평면으로 투영될 때의 점 대응 관계를 나타내는 2D 투영 변환이다. 회전, 확대/축소, 전단(shear), 이동, 왜곡을 모두 표현할 수 있는 변환이다.

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

여기서 변환 matrix 구성시에는 h_{33} 을 scale normalization하여 아래와 같이 구성한다.

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **Direct Linear Transformation (DLT)** : DLT는 3차원 세계 좌표 (X, Y, Z)를 이미지 평면상의 2차원 좌표 (x, y)로 변환하는 투영 모델로, 2D로 투영하는 과정이다. 이 과정에 있어 공선조건식을 기반으로 한 변환에 비해 실제 카메라의 물리적 특성을 더 잘 반영하지 못한다는 한계를 가지고 있다.

여기서 또한 변환 matrix 구성시에는 p_{34} 을 scale normalization하여 아래와 같이 구성한다.

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- **정규화 (Normalization)** : 입력 좌표들의 수치적 범위를 일정하게 조절하여 계산 과정의 안정성을 확보하고 오차를 최소화하는 방법이다.
- **공선조건식** : 공선조건식은 내부표정이 이미 완료되었다는 가정 하에서 공간상의 점과 센서(카메라) 중심, 이미지 평면상의 점이 하나의 직선상에 놓여 있다는 조건을 수식화한 모델이다. 이를 통해 EOP(Exterior Orientation Parameters)를 추정할 수 있다.

$$x = -f \cdot \frac{r_{11}(X - X_s) + r_{12}(Y - Y_s) + r_{13}(Z - Z_s)}{r_{31}(X - X_s) + r_{32}(Y - Y_s) + r_{33}(Z - Z_s)}$$

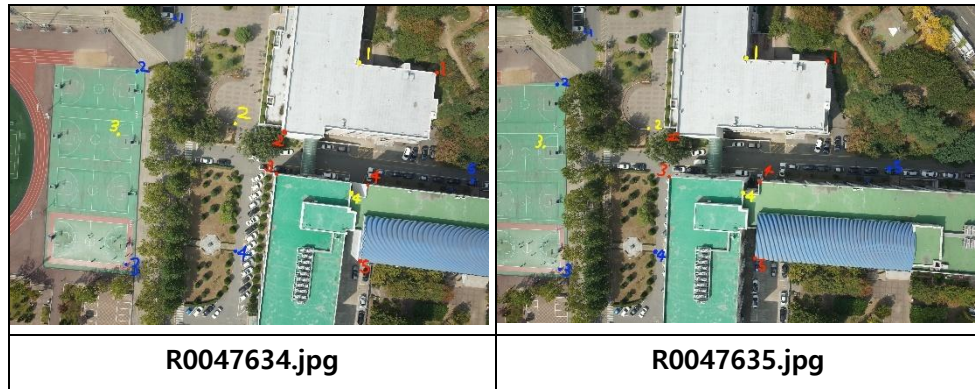
$$y = -f \cdot \frac{r_{21}(X - X_s) + r_{22}(Y - Y_s) + r_{23}(Z - Z_s)}{r_{31}(X - X_s) + r_{32}(Y - Y_s) + r_{33}(Z - Z_s)}$$

+ **공선조건식** 외부표정요소 $EOP = \{X_s, Y_s, Z_s, \Omega, \Phi, K\}$ 를 변수로 가지며 각각은 카메라의 XYZ 좌표, xyz축으로의 회전을 의미한다. 이때의 R은 회전행렬로 Ω, Φ, K 의 회전이 이루어졌을 때 나타나는 $R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$ 회전행렬을 사용한다.

3. Part1 Model Estimation Solution

1. 인접한 영상 두개를 선정하여 지붕면의 Point좌표 및 바닥면의 Point좌표를 확보하시오 (영상좌표)

지붕점, 바닥점이 많은 R0047634.jpg, R0047635.jpg 이미지를 선정



2. 두 영상에서 관측한 점들 중에서 같은 지점에 대한 영상점을 대응점으로 정의하시오

각 영상에서 동일한 지점을 식별하고 이를 대응점으로 정의

- (빨간점) 지붕 데이터셋: 5개 / (파란점) 바닥 데이터셋: 5개
- (노란점) 검증용 데이터셋: 지붕 2개 + 바닥 2개

3. 대응점들의 지붕면 데이터셋(A)과 바닥면 데이터셋(B)으로 구분하고 이를 다시 2개의 그룹으로 나누어 모델점(A1, B1)과 검사점(A2, B2)을 구분하시오 (모델점은 최소 5개 이상으로 추출)

- 지붕면 데이터셋 A → 모델점 A1 (5쌍), 검사점 A2 (2쌍)
- 바닥면 데이터셋 B → 모델점 B1 (5쌍), 검사점 B2 (2쌍)

[필요정보 정의 code in part1main]

```
string folder_dir = "C:/Users/tnddm/OneDrive/바탕 화면/codepaper/opencv/resource/";
string image_name1 = "pixel_R0047634.txt";
string image_name2 = "pixel_R0047635.txt";

cv::Mat_<double> A1, B1, A2, B2; // 모델점 데이터셋 세팅
cv::Mat_<double> A1t, B1t, A2t, B2t; // 대응점 데이터셋 세팅
readPoints(folder_dir + image_name1, A1, B1, A2, B2); // 모델점 읽기
readPoints(folder_dir + image_name2, A1t, B1t, A2t, B2t); // 대응점 읽기
```

[필요정보 정의 code readPoints function]

-모델 지붕점, 모델 바닥점, test지붕점, test바닥점을 각각의 행렬에 분류를 위한 함수

```
void readPoints(const string& filename,
cv::Mat_<double>& up,          // 지붕면 모델점 (roof model points)
cv::Mat_<double>& down,        // 바닥면 모델점 (floor model points)
cv::Mat_<double>& testup,      // 지붕면 테스트 모델점 (roof test model points)
cv::Mat_<double>& testdown,    // 바닥면 테스트 모델점 (floor test model points)
int un, int dn, int utn, int dtn) // 각 점들의 개수
{
    // 각 모델 점들을 위한 행렬 생성 (행 = 점 개수, 열 = 2D 좌표 [x, y])
    up.create(un, 2);
    down.create(dn, 2);
    testup.create(utn, 2);
    testdown.create(dtn, 2); // 메모리 반드시 할당 필요
    // 입력 파일 열기
    ifstream file(filename);
    if (!file.is_open()) {
        cerr << "파일 열기 실패: " << filename << endl;
        exit(-1); // 실패 시 종료
    }
    double x, y;
    int count = 0;
    // 파일에서 x, y 쌍을 읽고 순서에 따라 각 모델 점 행렬에 저장
    while (file >> x >> y) {
        if (count < un)
            // 지붕면 모델점
            up(count, 0) = x, up(count, 1) = y;
        else if (count < un + dn)
            // 바닥면 모델점
            down(count - un, 0) = x, down(count - un, 1) = y;
        else if (count < un + dn + utn)
            // 지붕면 테스트 모델점
            testup(count - un - dn, 0) = x, testup(count - un - dn, 1) = y;
        else if (count < un + dn + utn + dtn)
            // 바닥면 테스트 모델점
            testdown(count - un - dn - utn, 0) = x, testdown(count - un - dn - utn, 1) = y;

        count++;
    }
    file.close(); // 완료 후 파일 닫기
}
```

["pixel_R0047634.txt"]

```
4386.1 684.8
2810.045 1301.470
2743.365 1725.297
3679.915 1834.121
3597.732 2620.169
1694.770 124.635
```

1307.759 665.993
 1197.411 2646.145
 2311.411 2501.375
 4754.10 1781.54
 3586.024 590.192
 3503.988 1935.682
 2322.402 1213.669
 1108.993 1315.218

["pixel_R0047635.txt"]

3401.849 570.641
 1847.757 1329.632
 1769.3711 1754.6525
 2698.7390 1809.4652
 2663.517 2613.316
 908.705 255.600
 618.8049 812.4831
 651.6291 2704.6354
 1629.6686 2524.5145
 4055.771 1671.389
 2566.224 551.553
 2537.323 1930.194
 1550.272 1266.517
 486.334 1445.437

각각의 데이터셋은 그림판을 통해 직접 CR 좌표를 직접 추출

< 지붕면 모델점 데이터셋 A1,A1t >

측점	pixel_R0047634	pixel_R0047635
	COL ROW	COL ROW
1	4386.1 684.8	3401.849 570.641
2	2810.045 1301.470	1847.757 1329.632
3	2743.365 1725.297	1769.3711 1754.6525
4	3679.915 1834.121	2698.7390 1809.4652
5	3597.732 2620.169	2663.517 2613.316

< 바닥면 모델점 데이터셋 B1,B2t >

측점	pixel_R0047634	pixel_R0047635
	COL ROW	COL ROW
1	1694.770 124.635	908.705 255.600
2	1307.759 665.993	618.8049 812.4831
3	1197.411 2646.145	651.6291 2704.6354
4	2311.411 2501.375	1629.6686 2524.5145
5	4754.10 1781.54	4055.771 1671.389

< 지붕면 데이터셋 A2,A2t >

측점	pixel_R0047634	pixel_R0047635
	COL ROW	COL ROW
1	3586.024 590.192	2566.224 551.553
2	3503.988 1935.682	2537.323 1930.194

< 바닥면 데이터셋 B2,B2t >

1	2322.402 1213.669	1550.272 1266.517
2	1108.993 1315.218	486.334 1445.437

4. 두 영상의 변환관계를 Affine 변환으로 가정하고 Affine Parameter 를 추정하시오

4.1 두 영상 간의 좌표 변환을 Affine Transform 으로 가정한다면, 변환 관계는 다음과 같은 행렬 형태로 표현

a11,a12,a13,a21,a22, a23 는 Affine 변환의 계수로 각각 회전(rotation), 스케일(scale), 전단(shear) 및 **이동(translation)**을 조합한 결과이다

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

4.2 각 대응점 (x,y)→(x',y')에 대해 다음과 같은 선형식이 유도된다.

$$x' = a_{11} \cdot x + a_{12} \cdot y + a_{13}$$

$$y' = a_{21} \cdot x + a_{22} \cdot y + a_{23}$$

4.3 이를 여러 점에 대해 행렬식으로 정리하면 다음과 같다 - A×X = L

$$\begin{bmatrix} x1 & y1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x1 & y1 & 1 \\ x2 & y2 & 1 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ xn & yn & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & xn & yn & 1 \end{bmatrix} \times \begin{bmatrix} a11 \\ a12 \\ a13 \\ a21 \\ a22 \\ a23 \end{bmatrix} = \begin{bmatrix} x'1 \\ y'1 \\ x'2 \\ y'2 \\ \vdots \\ x'n \\ y'n \end{bmatrix}$$

A: 변수(x, y)에 해당하는 행렬

X: 추정할 Affine 계수 벡터

L: 대응 변수(x', y')를 열벡터로 구성한 행렬

변수(x, y)에는 R0047634.jpg 에서 선정한 점들에 대한 (Col, Row)좌표

(x', y')에는 R0047635.jpg 에서 대응하는 점들에 대한 (Col, Row)좌표

4.4 X 행렬을 알아내기 위한 최소제곱법 (행렬기반)

$X = (A^t \cdot A)^{-1} \cdot A^t \cdot L$ 식을 사용

이를 통해 affine transformation matrix 도출

코드에서는 이를 좀 더 안정적으로 풀어주는 cv::solve(A, L, X, cv::DECOMP_SVD); 사용

[Estimate affine matrix code]

- 두 영상 간의 대응점을 이용해 2D Affine 변환 행렬을 추정하는 함수
- 각 대응점을 기준으로 $AX=L$ 을 구성, 한 점당 두 개의 식을 생성하여 행 단위로 A 와 L 에 삽입
- cv::solve 함수의 SVD 분해를 통해 최소제곱 해를 구하고, 이로부터 Affine 변환 행렬을 반환

```
cv::Mat EstimateAffineMatrix(const cv::Mat_<double>& srcPoints, const cv::Mat_<double>&
dstPoints) {
    int N = srcPoints.rows;

    // 최소 3점 이상 필요하며, 입력은 2D 좌표여야 함
    CV_Assert(N >= 3 && srcPoints.cols == 2 && dstPoints.cols == 2);

    // A는 2N x 6 크기의 설계 행렬 (2D affine 변환의 6개의 unknown: a, b, c, d, e, f)
    cv::Mat_<double> A(2 * N, 6);

    // L은 관측값 벡터로, x', y' 좌표를 포함함
    cv::Mat_<double> L(2 * N, 1);

    for (int i = 0; i < N; ++i) {
        double x = srcPoints(i, 0), y = srcPoints(i, 1);
        double xp = dstPoints(i, 0), yp = dstPoints(i, 1);

        // x' = a*x + b*y + c 를 위한 방정식
        A(2 * i, 0) = x; A(2 * i, 1) = y; A(2 * i, 2) = 1;
        A(2 * i, 3) = 0; A(2 * i, 4) = 0; A(2 * i, 5) = 0;
        L(2 * i, 0) = xp;

        // y' = d*x + e*y + f 를 위한 방정식
        A(2 * i + 1, 0) = 0; A(2 * i + 1, 1) = 0; A(2 * i + 1, 2) = 0;
        A(2 * i + 1, 3) = x; A(2 * i + 1, 4) = y; A(2 * i + 1, 5) = 1;
        L(2 * i + 1, 0) = yp;
    }

    cv::Mat_<double> X;

    // 최소자승법으로 affine 변환의 계수 추정 ( $X = (A^T A)^{-1} A^T L$ )
    // SVD 분해로 해를 안정적으로 계산
    cv::solve(A, L, X, cv::DECOMP_SVD);

    // 3x3 homogeneous matrix 형태로 반환 (Affine 변환은 마지막 행이 [0 0 1])
    return (cv::Mat_<double>(3, 3) <<
        X(0), X(1), X(2),
        X(3), X(4), X(5),
        0, 0, 1);}
```

5. 모델점을 이용해 모델정확도와 검사점정확도를 산출하시오

- 모델정도 및 일반화 성능을 정량적으로 평가하기 위해 EvaluateRMSE 함수를 사용하였다. 이 함수는 추정된 변환 행렬(H)을 기반으로 점(srcModel)에 변환을 적용한 후, 실제 점(dstModel)과 비교하여 모델정확도를 산출한다.
- 각 점은 homogeneous 좌표(3×1)로 변환되어 행렬 곱을 통해 새로운 위치로 변환되고, 이후 정규화되어 2D 좌표로 복원된다.
- 예측된 위치와 실제 위치 간의 오차는 column 방향, row 방향으로 분해되어 각각의 제곱 평균 근사 오차(RMSE)로 표현되며, 이때 출력 항목은 다음과 같다:
 - **C-error RMSE**: 컬럼 방향 위치 오차
 - **R-error RMSE**: 로우 방향 위치 오차
 - **2D RMSE**: 전체 2차원 위치 오차 (벡터 합)
- 이를 통해 모델 점에 대해 계산된 RMSE는 변환 모델의 적합도를 나타내며, 검사 점에 대해 계산된 RMSE는 해당 모델의 일반화 성능을 평가한다.

[EvaluateRMSE function code]

-추정된 변환 행렬과 대응점 좌표를 입력받아, 변환된 점과 실제 점 간의 오차를 계산하고 컬럼(col) 방향, 로우(row) 방향, 전체 2차원 위치에 대한 RMSE(Root Mean Square Error)를 출력

```
// 변환행렬을 입력받고 그를 통해 구한 예측된 좌표 실제 좌표 사이의 RMSE (Root Mean Square Error)를 계산하고 출력

void EvaluateRMSE(const cv::Mat_<double>& H,
    const cv::Mat_<double>& srcTest,      // 변환 전 테스트 지점 (예: 지붕면 테스트 모델점)
    const cv::Mat_<double>& dstTest)     // 변환 후 비교 대상 지점 (예: 바닥면 테스트 모델점)
{
    int N = srcTest.rows;
    double sum_x_err = 0, sum_y_err = 0, sum_total = 0;

    // 출력 헤더
    cout << "\n===== Model Prediction Comparison =====\n";
    cout << "\nNo   GT_c   Pred_c   GT_r   Pred_r   dC   dR\n";
    cout << "-----\n";

    for (int i = 0; i < N; ++i) {
        // 테스트 입력 점을 homogeneous 좌표계로 확장 [x, y, 1]^T
        cv::Mat_<double> pt(3, 1);
        pt << srcTest(i, 0), srcTest(i, 1), 1.0;
```

```

// 변환 행렬 H를 적용하여 예측 위치 계산
cv::Mat_<double> pred = H * pt;
double pred_x = pred(0, 0) / pred(2, 0); // x 좌표 정규화
double pred_y = pred(1, 0) / pred(2, 0); // y 좌표 정규화

// Ground Truth (실측 위치)
double gt_x = dstTest(i, 0);
double gt_y = dstTest(i, 1);

// 예측값과 실제값 간의 오차
double dx = pred_x - gt_x;
double dy = pred_y - gt_y;

// 오차 제곱 합계 누적 (RMSE 계산을 위해)
sum_x_err += dx * dx;
sum_y_err += dy * dy;
sum_total += dx * dx + dy * dy;

// 포인트별 오차 출력: GT(정답), 예측, 오차
printf("%-3d %-8.2f %-8.2f %-8.2f %-8.2f %+6.2f %+6.2f\n",
        i + 1, gt_x, pred_x, gt_y, pred_y, dx, dy);
}

// RMSE 출력: col 방향, row 방향, 전체 2D 오차
cout << "\nC-error RMSE : " << sqrt(sum_x_err / N) << " px\n";
cout << "R-error RMSE : " << sqrt(sum_y_err / N) << " px\n";
cout << "2D RMSE      : " << sqrt(sum_total / N) << " px\n";
cout << "===== \n";
}

```

[EvaluateRMSE function code overloading]

-EvaluateRMSE 함수는 오버로딩되어 모델정확도와 검사점 정확도를 한번에 출력

```

// 모델점과 테스트점 각각에 대해 변환 정확도를 평가하는 오버로딩 함수
// 단일 변환 행렬 H를 기반으로 두 그룹(모델점 / 검사점)의 RMSE를 모두 출력함

void EvaluateRMSE(const cv::Mat_<double>& H,
    const cv::Mat_<double>& srcModel, // 변환 전 모델 점들
    const cv::Mat_<double>& dstModel, // 변환 후 실제 대응점
    const cv::Mat_<double>& srcTest,  // 변환 전 테스트 점들
    const cv::Mat_<double>& dstTest)  // 변환 후 실제 테스트 대응점
{
    // 모델 점 정확도 평가
    cout << "\n===== Model Point Accuracy =====\n";
    EvaluateRMSE(H, srcModel, dstModel);
    // 테스트 점 정확도 평가
    cout << "\n===== Test Point Accuracy =====\n";
    EvaluateRMSE(H, srcTest, dstTest);
}

```

a) A 데이터셋에 대해 4, 5 번 수행

- 지붕면 대응점 데이터셋 A 에서 모델점(A1)을 기반으로 Affine 변환 계수를 추정
- 추정된 계수로 Affine 변환 행렬 H 를 구성하였고, 이를 통해 모델점(A1)과 지붕검사점(A2)에 대해 예측 좌표를 계산
- 예측값과 실제값 간의 차이를 컬럼(col) 및 로우(row) 방향으로 나누어 RMSE 를 산출하여 모델 정확도 및 검사점 정확도를 평가

```
auto affineUp = EstimateAffineMatrix(A1, A1t);
PrintMatrixNicely(affineUp, "Affine Matrix (up)");
EvaluateRMSE(affineUp, A1, A1t, A2, A2t);
```

[A 데이터셋에 대한 모델정확도와 검사점정확도 결과]

```
=====
===== [ Affine Transformation ] =====
=====

[Affine Matrix (up)]
(   0.999498,   0.021347, -1000.320877 )
(  -0.075150,   1.023953,  202.022535 )
(   0.000000,   0.000000,   1.000000 )

===== Model Point Accuracy =====

===== Model Prediction Comparison =====

No  GT_c    Pred_c  GT_r    Pred_r   dC    dR
-----
1   3401.85  3398.20  570.64  573.61   -3.65  +2.97
2   1847.76  1836.10  1329.63 1323.49  -11.66  -6.14
3   1769.37  1778.50  1754.65 1762.48   +9.13  +7.83
4   2698.74  2716.90  1809.47 1803.53  +18.16  -5.93
5   2663.52  2651.54  2613.32 2614.59  -11.98  +1.27

C-error RMSE : 11.882096 px
R-error RMSE : 5.378452 px
2D RMSE      : 13.042698 px

=====

===== Test Point Accuracy =====

===== Model Prediction Comparison =====

No  GT_c    Pred_c  GT_r    Pred_r   dC    dR
-----
1   2566.22  2596.50  551.55  536.86  +30.28 -14.69
2   2537.32  2543.23  1930.19 1920.75   +5.91  -9.45

C-error RMSE : 21.814043 px
R-error RMSE : 12.349523 px
2D RMSE      : 25.067174 px
```

b) B 데이터셋에 대해 4, 5 번 수행

바닥면 대응점 데이터셋 B에 대해서도 동일한 방식으로 진행

```
auto affineDown = EstimateAffineMatrix(B1, B1t);
PrintMatrixNicely(affineDown, "Affine Matrix (down)");
EvaluateRMSE(affineDown, B1, B1t, B2, B2t);
```

[B 데이터셋에 대한 모델정확도와 검사점정확도 결과]

```
=====
[Affine Matrix (down)]
(   0.977564,   0.065389,  -730.580818 )
(  -0.058918,   0.962056,   241.117452 )
(   0.000000,   0.000000,   1.000000 )

===== Model Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	908.71	934.32	255.60	261.17	+25.61	+5.57
2	618.80	591.39	812.48	804.79	-27.42	-7.69
3	651.63	612.99	2704.64	2716.31	-38.64	+11.67
4	1629.67	1692.53	2524.51	2511.40	+62.86	-13.12
5	4055.77	4033.35	1671.39	1674.96	-22.42	+3.57

```

C-error RMSE : 38.353936 px
R-error RMSE : 9.069571 px
2D RMSE      : 39.411693 px

=====

===== Test Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	1550.27	1619.08	1266.52	1271.90	+68.80	+5.39
2	486.33	439.53	1445.44	1441.09	-46.80	-4.35

```

C-error RMSE : 58.841168 px
R-error RMSE : 4.894235 px
2D RMSE      : 59.044361 px
=====
```

c) A, B 데이터셋을 통합하여 4, 5 번 수행 및 정확도 비교

바닥면과 지붕면을 통합한 대응점 데이터셋에 대해서도 동일한 방식으로 진행

```
cv::Mat_<double> AB1, AB2, AB1t, AB2t;
cv::vconcat(A1, B1, AB1);
cv::vconcat(A1t, B1t, AB1t);
cv::vconcat(A2, B2, AB2);
cv::vconcat(A2t, B2t, AB2t);
auto affineAll = EstimateAffineMatrix(AB1, AB1t);
PrintMatrixNicely(affineAll, "Affine Matrix (updown)");
EvaluateRMSE(affineAll, AB1, AB1t, AB2, AB2t);
```

[통합 데이터셋에 대한 모델정확도와 검사점정확도 결과]

```
=====
[Affine Matrix (updown)]
(   0.926558,    0.048480,  -691.510832 )
(  -0.063382,    0.981948,   225.283309 )
(   0.000000,    0.000000,    1.000000 )

===== Model Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	3401.85	3405.66	570.64	619.72	+3.82	+49.08
2	1847.76	1975.25	1329.63	1325.15	+127.50	-4.48
3	1769.37	1934.02	1754.65	1745.56	+164.65	-9.10
4	2698.74	2807.06	1809.47	1793.05	+108.32	-16.41
5	2663.52	2769.02	2613.32	2570.12	+105.51	-43.19
6	908.71	884.83	255.60	240.25	-23.87	-15.35
7	618.80	552.49	812.48	796.37	-66.31	-16.12
8	651.63	546.24	2704.64	2747.77	-105.38	+43.13
9	1629.67	1571.41	2524.51	2535.00	-58.26	+10.49
10	4055.77	3799.81	1671.39	1673.34	-255.96	+1.95

```

C-error RMSE : 122.974896 px
R-error RMSE : 26.676189 px
2D RMSE      : 125.834988 px

=====

===== Test Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	2566.22	2659.76	551.55	577.53	+93.54	+25.98
2	2537.32	2648.98	1930.19	1903.93	+111.66	-26.26
3	1550.27	1519.17	1266.52	1269.84	-31.10	+3.33
4	486.33	399.80	1445.44	1446.47	-86.54	+1.03

```

C-error RMSE : 86.128338 px
R-error RMSE : 18.551836 px
2D RMSE      : 88.103696 px
=====
```

[고찰] A, B 를 합한 데이터 셋에서 추정한 affine 변환이 정확도가 가장 낮게 나타났다.
그 이유에 대해 분석해보았다.

Affine 변환은 하나의 평면 내에서의 선형 관계를 가정하므로, 서로 다른 평면에서 온 점들을 하나의 Affine 모델로 설명할 경우, 오차가 증가할 수밖에 없음.

- Affine 변환은 회전, 이동, 스케일, 전단 등만 표현 가능하지만, 투영, 평행선의 왜곡은 반영하지 못함.
- Affine 모델은 지붕과 바닥 같은 깊이차나 투영 변화를 설명할 수 없어 통합 데이터 셋 사용시 오차 증가로 이어짐

6. 두 영상의 변환관계를 Homography 변환으로 가정하고 Parameter를 추정하시오

6.1 두 영상 간의 좌표 변환을 Homography Transform 으로 가정한다면, 변환 관계는 다음과 같은 행렬 형태로 표현 $h_{33}=1$ 고정하여 자유도를 8 개로 축소

이때 $h_{11}, h_{12}, \dots, h_{33}$ 는 Homography 변환의 계수들로, 회전(rotation), 스케일(scale), 전단(shear), 이동(translation) 뿐만 아니라 시점 변화(viewpoint change) 및 원근 왜곡(perspective distortion)을 포함한 개념

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

6.2 각 대응점 $(x, y) \rightarrow (x', y')$ 에 대해 다음과 같은 비선형식이 유도

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}$$

6.3 이를 선형식으로 바꾸기 위해 양변에 분모를 곱

$$x'(h_{31}x + h_{32}y + 1) = h_{11}x + h_{12}y + h_{13}$$

$$y'(h_{31}x + h_{32}y + 1) = h_{21}x + h_{22}y + h_{23}$$

$$h_{11}x + h_{12}y + h_{13} - x'(h_{31}x + h_{32}y) = x'$$

$$h_{21}x + h_{22}y + h_{23} - y'(h_{31}x + h_{32}y) = y'$$

6.4 이를 여러 점에 대해 행렬식으로 정리 - $A \times X = L$

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1 * x'_1 & -y_1 * x'_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1 * y'_1 & -y_1 * y'_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2 * x'_2 & -y_2 * x'_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_n & y_n & 1 & 0 & 0 & 0 & -x_n * x'_n & -y_n * x'_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -x_n * y'_n & -y_n * y'_n \end{bmatrix} \times \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x'_1 \\ y'_1 \\ x'_2 \\ y'_2 \\ \vdots \\ x'_n \\ y'_n \end{bmatrix}$$

A: 변수 (x, y) 에 해당하는 행렬

X: 추정할 homography 계수 벡터

L: 대응 변수 (x', y') 를 열벡터로 구성한 행렬

변수 (x, y) 에는 R0047634.jpg 에서 선정한 점들에 대한 (Col, Row)좌표

(x', y') 에는 R0047635.jpg 에서 대응하는 점들에 대한 (Col, Row)좌표

6.5 X 행렬을 알아내기 위한 최소제곱법 (행렬기반)

$X = (A^T \cdot A)^{-1} \cdot A^T \cdot L$ 식을 사용

이를 통해 homography transformation matrix 도출

코드에서는 안정적으로 풀어주는 `cv::solve(A, L, X, cv::DECOMP_SVD);` 를 사용

[Estimate homography matrix code]

-두 영상 간의 대응점을 이용해 2D Homography 변환 행렬을 추정하는 함수

-각 대응점을 기준으로 $AX=L$ 을 구성, 한 점당 두 개의 식을 생성하여 행 단위로 A 와 L 에 삽입

-`cv::solve` 함수의 SVD 분해를 통해 최소제곱 해를 구하고, 이로부터 Homography 변환 행렬을 반환

```
cv::Mat EstimateHomographyMatrix(const cv::Mat_<double>& srcPoints, const
cv::Mat_<double>& dstPoints) {
    int N = srcPoints.rows;
    // 최소 4쌍 이상의 대응점 필요
    CV_Assert(N >= 4 && srcPoints.cols == 2 && dstPoints.cols == 2);
    // 설계행렬 A (2N x 8), 관측벡터 L (2N x 1)
    cv::Mat_<double> A(2 * N, 8);
    cv::Mat_<double> L(2 * N, 1);

    for (int i = 0; i < N; ++i) {
        double x = srcPoints(i, 0), y = srcPoints(i, 1);
        double xp = dstPoints(i, 0), yp = dstPoints(i, 1);
        // x' = (h11*x + h12*y + h13) / (h31*x + h32*y + 1)
        A(2 * i, 0) = x;
        A(2 * i, 1) = y;
        A(2 * i, 2) = 1;
        A(2 * i, 3) = 0;
        A(2 * i, 4) = 0;
        A(2 * i, 5) = 0;
        A(2 * i, 6) = -x * xp; // 비선형성 선형화
        A(2 * i, 7) = -y * xp;
        L(2 * i, 0) = xp;
        // y' = (h21*x + h22*y + h23) / (h31*x + h32*y + 1)
        A(2 * i + 1, 0) = 0;
        A(2 * i + 1, 1) = 0;
        A(2 * i + 1, 2) = 0;
        A(2 * i + 1, 3) = x;
        A(2 * i + 1, 4) = y;
        A(2 * i + 1, 5) = 1;
        A(2 * i + 1, 6) = -x * yp;
        A(2 * i + 1, 7) = -y * yp;
        L(2 * i + 1, 0) = yp;
    }
    // 최소자승법을 통한 해 구하기 (SVD 이용)
    cv::Mat_<double> X;
    cv::solve(A, L, X, cv::DECOMP_SVD);
    // Homography 행렬 H (3x3): 마지막 원소는 scale 고정 (h33 = 1)
```

```
return (cv::Mat_<double>(3, 3) <<
        X(0), X(1), X(2),
        X(3), X(4), X(5),
        X(6), X(7), 1.0);}
```

7. 모델점을 이용해 모델정확도와 검사점정확도를 산출하시오 (a, b, c 동일하게 수행)

a) A 데이터셋에 대해 4, 5 번 수행

- 지붕면 대응점 데이터셋 A에서 모델점(A1)을 기반으로 Homography 변환 계수를 추정
- 각 대응점에 대해 $AX=L$ 형태의 선형 시스템을 구성하고, `cv::solve` 함수를 통해 계수 X 를 추정, 추정된 계수로 변환 행렬 구성, 이를 통해 모델점(A1)과 지붕검사점(A2)에 대해 예측 좌표를 계산
- Affine 변환의 정확도 산출 방식과 같은 방식을 수행

```
auto homoUp = EstimateHomographyMatrix(A1, A1t);
PrintMatrixNicely(homoUp, "Homography Matrix (up)");
EvaluateRMSE(homoUp, A1, A1t, A2, A2t);
```

[A 데이터셋에 대한 모델정확도와 검사점정확도 결과]

```
=====
===== [ Homography Transformation ] =====
=====

[Homography Matrix (up)]
(  0.745543,  -0.003443,  -494.497203 )
( -0.120383,   0.820149,  432.300088 )
( -0.000040,  -0.000014,   1.000000 )

===== Model Point Accuracy =====

===== Model Prediction Comparison =====

No  GT_c    Pred_c    GT_r    Pred_r    dC    dR
-----
1   3401.85  3401.22  570.64  571.45    -0.63  +0.80
2   1847.76  1834.51  1329.63 1334.95   -13.25  +5.32
3   1769.37  1781.99  1754.65 1749.91   +12.62  -4.75
4   2698.74  2708.58  1809.47 1803.79    +9.84  -5.67
5   2663.52  2654.93  2613.32 2617.61    -8.59  +4.30

C-error RMSE : 10.057807 px
R-error RMSE : 4.519618 px
2D RMSE      : 11.026623 px

=====

===== Test Point Accuracy =====

===== Model Prediction Comparison =====

No  GT_c    Pred_c    GT_r    Pred_r    dC    dR
-----
1   2566.22  2565.35  551.55  571.10    -0.87  +19.55
2   2537.32  2532.45  1930.19 1916.87    -4.88  -13.33

C-error RMSE : 3.503133 px
R-error RMSE : 16.730425 px
2D RMSE      : 17.093247 px
```

b) B 데이터셋에 대해 4, 5 번 수행

바닥면 대응점 데이터셋 B 에 대해서도 동일한 방식으로 진행

```
auto homoDown = EstimateHomographyMatrix(B1, B1t);
PrintMatrixNicely(homoDown, "Homography Matrix (down)");
EvaluateRMSE(homoDown, B1, B1t, B2, B2t);
```

[B 데이터셋에 대한 모델정확도와 검사점정확도 결과]

```
=====
[Homography Matrix (down)]
(   0.790081,   0.062067, -488.136229 )
(  -0.113650,   0.903384,  321.397078 )
(  -0.000034,  -0.000002,   1.000000 )

===== Model Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	908.71	912.00	255.60	256.39	+3.30	+0.79
2	618.80	614.85	812.48	811.94	-3.95	-0.54
3	651.63	652.16	2704.64	2700.04	+0.54	-4.59
4	1629.67	1630.42	2524.51	2531.26	+0.75	+6.74
5	4055.77	4055.13	1671.39	1668.96	-0.64	-2.43

```

C-error RMSE : 2.356605 px
R-error RMSE : 3.831682 px
2D RMSE      : 4.498375 px

=====

===== Test Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	1550.27	1549.32	1266.52	1257.11	-0.95	-9.41
2	486.33	489.54	1445.44	1441.99	+3.21	-3.45

```

C-error RMSE : 2.366866 px
R-error RMSE : 7.086817 px
2D RMSE      : 7.471615 px

=====
```

c) A, B 데이터셋을 통합하여 4, 5 번 수행 및 정확도 비교

바닥면과 지붕면을 통합한 대응점 데이터셋에 대해서도 동일한 방식으로 진행

```
auto homoAll = EstimateHomographyMatrix(AB1, AB1t);
PrintMatrixNicely(homoAll, "Homography Matrix (updown)");
EvaluateRMSE(homoAll, AB1, AB1t, AB2, AB2t);
```

[통합 데이터셋에 대한 모델정확도와 검사점정확도 결과]

```
=====
[Homography Matrix (updown)]
(   0.613850,   0.039487, -254.371932 )
(  -0.139813,   0.807529,  405.830037 )
(  -0.000061,  -0.000011,   1.000000 )

===== Model Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	3401.85	3400.03	570.64	476.67	-1.82	-93.97
2	1847.76	1869.30	1329.63	1306.72	+21.54	-22.91
3	1769.37	1841.09	1754.65	1739.96	+71.72	-14.69
4	2698.74	2750.13	1809.47	1817.26	+51.39	+7.79
5	2663.52	2738.10	2613.32	2686.36	+74.58	+73.05
6	908.71	883.36	255.60	301.04	-25.34	+45.44
7	618.80	629.56	812.48	833.43	+10.75	+20.94
8	651.63	652.00	2704.64	2646.62	+0.37	-58.02
9	1629.67	1519.83	2524.51	2529.64	-109.84	+5.12
10	4055.77	3960.78	1671.39	1709.00	-94.99	+37.61

```

C-error RMSE : 59.716226 px
R-error RMSE : 47.199960 px
2D RMSE      : 76.117434 px

=====

===== Test Point Accuracy =====

===== Model Prediction Comparison =====
```

No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	2566.22	2542.74	551.55	491.79	-23.48	-59.77
2	2537.32	2579.69	1930.19	1933.87	+42.37	+3.67
3	1550.27	1442.97	1266.52	1256.02	-107.30	-10.50
4	486.33	521.20	1445.44	1430.56	+34.87	-14.87

```

C-error RMSE : 61.392485 px
R-error RMSE : 31.293104 px
2D RMSE      : 68.907878 px
=====
```

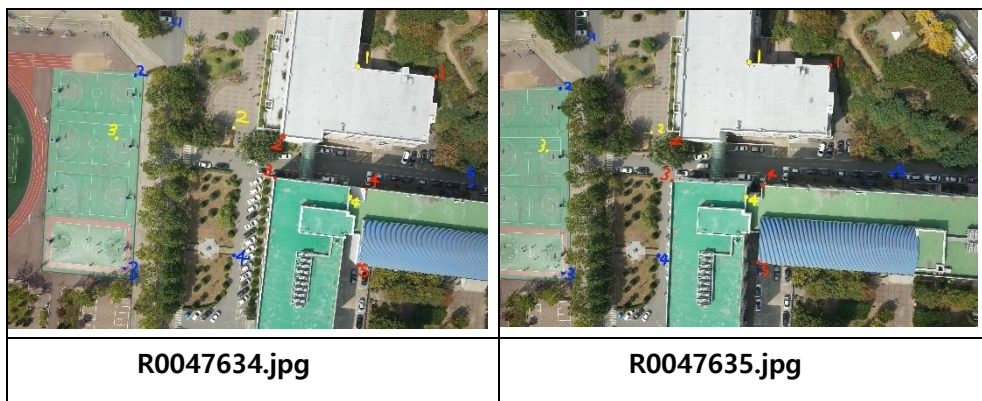
[고찰]A,B 를 합한 데이터 셋에서 추정한 homography 변환이 정확도가 가장 낮게 나타났다. 그 이유에 대해 분석해보았다.

Homography 변환 역시 하나의 평면 내에서의 변환 관계를 가정하므로, 서로 다른 평면에서 온 점들을 하나의 모델로 설명할 경우, 오차가 증가할 수밖에 없음.

- 지붕과 바닥은 카메라에 대해 서로 다른 거리 및 각도를 갖기 때문에, 같은 Homography 행렬로 투영할 경우 왜곡이 심화됨.
- Homography 는 투영변환을 잘 처리하지만, 다수 평면에서 발생하는 복합 투영은 처리할 수 없음.

8. Affine 모델의 정확도와 Homography 모델의 정확도를 비교하고 그 결과를 고찰하시오

- Affine 변환에서의 정확도가 Homography모델의 정확도보다 전반적으로 낮게 나타났다. 그 이유를 분석하니 두 변환에서의 차이는 원근 왜곡의 보정 여부, 평행선의 유지의 차이가 두드러진다고 생각했다.



- Affine 변환은 카메라와 평행한 단일 평면에 대해 변형을 단순화하여 처리할 수 있기 때문에, 넓고 평평한 구조에는 적합하다. 하지만 원근감이나 기하학적 왜곡을 반영하지 못하므로, 기울어졌거나 카메라 시점이 다른 평면과 함께 사용할 경우 정확도가 떨어진다. 이 점에 있어 선정한 두 영상을 보면 지표 평면의 각도가 약간 다른 것을 확인할 수 있다. 반면, Homography 변환은 평행선의 틀어짐을 수학적으로 모델링할 수 있어, 평면의 기울기로 인해 발생하는 투영 왜곡도 Homography는 수학적으로 포함하기 때문에, 두 영상에서 평면의 각도가 약간 다르더라도 이를 보정하고 보다 정확한 매칭을 수행할 수 있다고 판단하였다.
- 특히 바닥면과 지붕면을 함께 다룰 때에도, Homography는 각 평면의 기하학적 변형을 더 잘 반영할 수 있기 때문에 보다 정밀한 대응점 정합이 가능하다. Affine 변환은 단순 선형 변환이기 때문에, 서로 다른 깊이를 가진 복수 평면을 동시에 모델링할 경우에는 한계가 발생한다. 따라서 통합 데이터셋에서는 Affine보다는 Homography가 더 높은 자유도를 바탕으로 보다 정확한 fitting을 수행할 수 있다고 판단하였다.

4. Part2 DLT Model Estimation Solution

1. 최소 7개 이상의 GCP를 가지는 영상 하나를 선정하시오

===== GCP Information =====

1. Image Name : R0047633.JPG

2. Number of GCPs : 12

No	X	Y	Z	Col	Row

1	292315.611	4147477.919	37.778	3391	2018
2	292273.541	4147502.668	37.531	3479	595
3	292298.234	4147376.305	34.710	721	2816
4	292282.672	4147366.160	34.216	304	2567
5	292279.628	4147393.802	33.749	887	2170
6	292224.581	4147422.446	33.758	836	474
7	292212.502	4147433.762	33.792	946	20
8	292262.128	4147467.824	33.848	2426	796
9	292252.361	4147441.385	33.829	1643	903
10	292269.244	4147432.765	33.845	1654	1433
11	292301.149	4147432.223	34.005	2058	2219
12	292328.515	4147431.379	34.542	2385	2886

12개의 gcp를 가지는 R0047633.JPG를 선정

R0047633.JPG에 대한 GCP, EOP, IOP를 모두 읽고 벡터에 저장

[필요정보 정의 EOP, IOP, GCP 읽기]

```
string folder_dir = "C:/Users/tnddm/OneDrive/바탕 화면/codepaper/opencv/resource/";
string image_name = "R0047633.JPG";
string IOP_name = folder_dir + "IOFile_Metric.txt";
string EOP_name = folder_dir + "input_EOP.txt";
string GCP_name = folder_dir + "GCP.txt";

// 2. 파일 읽기
IOP ImageIOP;
vector<EOP> ImageEOPs;
EOP ImageEOP;
vector<GCP> ImageGCPs;
vector<GCP> ImageGCP;

ReadIOP(IOP_name, ImageIOP);
ReadEOP(EOP_name, ImageEOPs);
ReadGCP(GCP_name, ImageGCPs);
ReadImageEOP(ImageEOPs, image_name, ImageEOP);
ReadImageGCP(ImageGCPs, image_name, ImageGCP);

// 3. GCP 정보 출력
PrintGCPInfo(ImageGCP, image_name);
```

[ReadImageEOP function]

-특정 image에 해당하는 EOP 읽기

```
int ReadImageEOP(std::vector<EOP> in_eop_vec, string in_image_name, EOP&
out_eop)
{
    int result = -1;
    //=====
    // EOP vector중 입력 영상에 해당하는 EOP 불러오기
    int image_index;
    for (int i = 0; i < in_eop_vec.size(); i++) {
        if (in_image_name == in_eop_vec[i].filename) {
            image_index = i;
            out_eop = in_eop_vec[i];

            result = 0;
        }
    }
    return result;
}
```

[ReadImageEOP function]

-특정 image에 존재하는 GCP 읽기

```
int ReadImageGCP(std::vector<GCP> in_gcp_vec, string in_image_name,
std::vector<GCP>& out_gcp_vec)
```

```

{
    int result = 0;
    out_gcp_vec.resize(0);
    for (int i = 0; i < in_gcp_vec.size(); i++) {
        GCP one_GCP;
        one_GCP.order.resize(1);
        one_GCP.paircount = 1;
        int pairCount = in_gcp_vec[i].paircount;
        for (int j = 0; j < pairCount; j++) {
            if (in_gcp_vec[i].order[j].ImageName == in_image_name) {
                one_GCP.xyz.X = in_gcp_vec[i].xyz.X;
                one_GCP.xyz.Y = in_gcp_vec[i].xyz.Y;
                one_GCP.xyz.Z = in_gcp_vec[i].xyz.Z;
                one_GCP.order[0] = in_gcp_vec[i].order[j];
                out_gcp_vec.push_back(one_GCP);
            }
        }
    }

    if (out_gcp_vec.size() == 0) result = -1;
    else result = 0;

    return result;
}

```

2. 그 중 6개의 GCP 정보를 추출하여 모델링에 사용하고 나머지 점은 검사점으로 사용하시오 - 앞에서부터 6개를 모델점, 남은 점들을 검사점으로 사용

```

// 4. 모델점/검사점 분리 (앞 6개 모델링)
vector<GCP> ModelGCP;
vector<GCP> CheckGCP;
for (int i = 0; i < ImageGCP.size(); ++i) {
    if (i < 6) ModelGCP.push_back(ImageGCP[i]);
    else CheckGCP.push_back(ImageGCP[i]);
}

```

3. 영상좌표로 바로 변환하는 DLT 모델을 추정하시오 (X,Y,Z) to (c,r)

3.1 영상좌표로 바로 변환하기 위해 DLT 모델을 다음과 같은 행렬 형태로 표현할 수 있다. 보통 $p_{34}=1$ 고정하여 자유도를 11 개로 축소

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3.2 각 GCP $(x,y,z) \rightarrow (x',y')$ 에 대해 다음과 같은 비선형식이 유도

$$x' = \frac{p_{11}x + p_{12}y + p_{13}z + p_{14}}{p_{31}x + p_{32}y + p_{33}z + 1}, \quad y' = \frac{p_{21}x + p_{22}y + p_{23}z + p_{24}}{p_{31}x + p_{32}y + p_{33}z + 1}$$

3.3 이를 선형식으로 바꾸기 위해 양변에 분모를 곱

$$\begin{aligned}x'(p_{31}x+p_{32}y+p_{33}z+1)&=p_{11}x+p_{12}y+p_{13} \\y'(p_{31}x+p_{32}y+p_{33}z+1)&=p_{21}x+p_{22}y+p_{23}z+p_{24} \\p_{11}x+p_{12}y+p_{13}z+p_{14}-x'(p_{31}x+p_{32}y+p_{33}z)&=x' \\p_{21}x+p_{22}y+p_{23}z+p_{24}-y'(p_{31}x+p_{32}y+p_{33}z)&=y'\end{aligned}$$

3.4 이를 여러 점에 대해 행렬식으로 정리 - $A \times X = L$

$$\begin{bmatrix}x_1 & y_1 & z_1 & 1 & 0 & 0 & 0 & 0 & -x_1 * x'_1 & -y_1 * x'_1 & -z_1 * x'_1 \\0 & 0 & 0 & 0 & x_1 & y_1 & z_1 & 1 & -x_1 * y'_1 & -y_1 * y'_1 & -z_1 * y'_1 \\x_2 & y_2 & z_2 & 1 & 0 & 0 & 0 & 0 & -x_2 * x'_2 & -y_2 * x'_2 & -z_2 * x'_2 \\\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\x_n & y_n & z_n & 1 & 0 & 0 & 0 & 0 & -x_n * x'_n & -y_n * x'_n & -z_n * x'_n \\0 & 0 & 0 & 0 & x_n & y_n & z_n & 1 & -x_n * y'_n & -y_n * y'_n & -z_n * y'_n\end{bmatrix} \begin{bmatrix}p_{11} \\p_{12} \\p_{13} \\p_{14} \\p_{21} \\p_{22} \\p_{23} \\p_{24} \\p_{31} \\p_{32} \\p_{33}\end{bmatrix} = \begin{bmatrix}x'_1 \\y'_1 \\x'_2 \\y'_2 \\\vdots \\x'_n \\y'_n\end{bmatrix}$$

A: 변수(x, y, z)에 해당하는 행렬

X: 추정할 DLT 계수 벡터

L: 대응 변수(x', y')를 열벡터로 구성한 행렬

-변수(x, y, z)에는 R0047633.jpg 에 나타난 GCP 의 3 차원 좌표

-(x', y')에는 대응하는 GCP 들에 대한 (Col, Row)좌표

3.5 X 행렬을 알아내기 위한 최소제곱법 (행렬기반)

$X = (A^T \cdot A)^{-1} \cdot A^T \cdot L$ 식을 사용

이를 통해 homography transformation matrix 도출

코드에서는 cv::solve(A, L, X, cv::DECOMP_QR); 를 사용

[Estimate DLT matrix code XYZ -> CR]

- GCP 이용해 3 차원 xyz 좌표를 바로 Col Row 로 변환하는 함수
- 한 GCP 당 2 개의 식을 생성하여 행 단위로 A 와 L 에 삽입
- cv::solve 함수의 QR 를 통해 최소제곱 해를 구하고, 이로부터 DLT 행렬을 반환

```
cv::Mat EstimateDLT_Model_XYZtoCR(const std::vector<GCP>& gcps) {
    int n = gcps.size();
    cv::Mat_<double> XYZ(n, 3), CR(n, 2);

    // GCP에서 3D(XYZ) 좌표와 대응되는 영상 좌표(Col, Row) 추출
    for (int i = 0; i < n; ++i) {
        XYZ(i, 0) = gcps[i].xyz.X;
        XYZ(i, 1) = gcps[i].xyz.Y;
        XYZ(i, 2) = gcps[i].xyz.Z;

        CR(i, 0) = gcps[i].order[0].CR.Col;
        CR(i, 1) = gcps[i].order[0].CR.Row;
    }
}
```

```

// A 행렬 (2n x 11) 및 관측 벡터 L (2n x 1) 생성
cv::Mat_<double> A(2 * n, 11);
cv::Mat_<double> L(2 * n, 1);

for (int i = 0; i < n; ++i) {
    double X = XYZ(i, 0), Y = XYZ(i, 1), Z = XYZ(i, 2);
    double c = CR(i, 0), r = CR(i, 1);

    // 첫 번째 식
    A(2 * i, 0) = X;
    A(2 * i, 1) = Y;
    A(2 * i, 2) = Z;
    A(2 * i, 3) = 1;
    A(2 * i, 4) = 0; A(2 * i, 5) = 0; A(2 * i, 6) = 0; A(2 * i, 7) = 0;
    A(2 * i, 8) = -c * X;
    A(2 * i, 9) = -c * Y;
    A(2 * i, 10) = -c * Z;
    L(2 * i, 0) = c;

    // 두 번째 식
    A(2 * i + 1, 0) = 0; A(2 * i + 1, 1) = 0; A(2 * i + 1, 2) = 0; A(2 * i + 1, 3) = 0;
    A(2 * i + 1, 4) = X;
    A(2 * i + 1, 5) = Y;
    A(2 * i + 1, 6) = Z;
    A(2 * i + 1, 7) = 1;
    A(2 * i + 1, 8) = -r * X;
    A(2 * i + 1, 9) = -r * Y;
    A(2 * i + 1, 10) = -r * Z;
    L(2 * i + 1, 0) = r;
}

// 최소자승 해 구하기 (QR 분해 방식)
cv::Mat Xsol;
cv::solve(A, L, Xsol, cv::DECOMP_QR);
Xsol.convertTo(Xsol, CV_64F); // 안정성을 위한 자료형 고정

// DLT 행렬 (3x4) 구성
cv::Mat_<double> DLT(3, 4);
DLT(0, 0) = Xsol.at<double>(0, 0);
DLT(0, 1) = Xsol.at<double>(1, 0);
DLT(0, 2) = Xsol.at<double>(2, 0);
DLT(0, 3) = Xsol.at<double>(3, 0);

DLT(1, 0) = Xsol.at<double>(4, 0);
DLT(1, 1) = Xsol.at<double>(5, 0);
DLT(1, 2) = Xsol.at<double>(6, 0);
DLT(1, 3) = Xsol.at<double>(7, 0);

```

```

DLT(2, 0) = Xsol.at<double>(8, 0);
DLT(2, 1) = Xsol.at<double>(9, 0);
DLT(2, 2) = Xsol.at<double>(10, 0);
DLT(2, 3) = 1.0; // scale 고정

return DLT;
}

```

4. GCP의 c,r좌표를 카메라 좌표계로 변환하여 (X,Y,Z) to (x,y)로 변환시키는 DLT 모델을 추정하시오

4.1 카메라좌표계로 바로 변환하기 위해 DLT 모델을 다음과 같은 행렬 형태로 표현할 수 있다. 보통 $p_{34}=1$ 고정하여 자유도를 11 개로 축소

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

4.2 같은 방식으로 $A \cdot X = L$ 행렬식을 구성

$$\begin{bmatrix} x1 & y1 & z1 & 1 & 0 & 0 & 0 & 0 & -x1 * x'1 & -y1 * x'1 & -z1 * x'1 \\ 0 & 0 & 0 & 0 & x1 & y1 & z1 & 1 & -x1 * y'1 & -y1 * y'1 & -z1 * y'1 \\ x2 & y2 & z2 & 1 & 0 & 0 & 0 & 0 & -x2 * x'2 & -y2 * x'2 & -z2 * x'2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ xn & yn & zn & 1 & 0 & 0 & 0 & 0 & -xn * x'n & -yn * x'n & -zn * x'n \\ 0 & 0 & 0 & 0 & xn & yn & zn & 1 & -xn * y'n & -yn * y'n & -zn * y'n \end{bmatrix} \begin{bmatrix} p11 \\ p12 \\ p13 \\ p14 \\ p21 \\ p22 \\ p23 \\ p24 \\ p31 \\ p32 \\ p33 \end{bmatrix} = \begin{bmatrix} x'1 \\ y'1 \\ x'2 \\ y'2 \\ \vdots \\ x'n \\ y'n \end{bmatrix}$$

A: 변수(x, y, z)에 해당하는 행렬

X: 추정할 DLT 계수 벡터

L: 대응 변수(x', y')를 열벡터로 구성한 행렬

- 변수(x, y, z)에는 R0047633.jpg 에 나타난 GCP 의 3 차원 좌표

- (x', y')에는 대응하는 GCP 들에 대한 (Col, Row)좌표를 이미지좌표로 변환한 좌표

4.3 X 행렬을 알아내기 위한 최소제곱법 (행렬기반)

3.과 같은 방식 풀이

[Estimate DLT matrix code XYZ -> XY]

- GCP 이용해 3 차원 xyz 좌표를 바로 이미지좌표로 변환하는 함수
- 한 GCP 당 2 개의 식을 생성하여 행 단위로 A 와 L 에 삽입
- 이후 cv::solve 함수의 SVD 분해를 통해 최소제곱 해를 구하고, 이로부터 DLT 행렬을 반환

```

cv::Mat EstimateDLT_Model_XYZtoXY(const vector<GCP>& gcps, const IOP& iop) {
    int N = gcps.size();
}

```

```

cv::Mat A(2 * N, 11, CV_64F);
cv::Mat L(2 * N, 1, CV_64F);

// 이미지 중심 (principal point) 및 셀 크기 (pixel size)
double cx = iop.ColSize / 2.0;
double cy = iop.RowSize / 2.0;
double s = iop.CellSize; // CellSize = s_x = s_y (등방성 가정)

for (int i = 0; i < N; ++i) {
    double X = gcps[i].xyz.X;
    double Y = gcps[i].xyz.Y;
    double Z = gcps[i].xyz.Z;
    double c = gcps[i].order[0].CR.Col;
    double r = gcps[i].order[0].CR.Row;

    // 사진 좌표계로 변환: 이미지 중심 기준 + 셀 크기 보정
    double xprime = (c - cx) * s;
    double yprime = -(r - cy) * s; // 좌표계 y 방향 반전

    // 첫 번째 식:
    A.at<double>(2 * i, 0) = X;
    A.at<double>(2 * i, 1) = Y;
    A.at<double>(2 * i, 2) = Z;
    A.at<double>(2 * i, 3) = 1;
    A.at<double>(2 * i, 4) = 0;
    A.at<double>(2 * i, 5) = 0;
    A.at<double>(2 * i, 6) = 0;
    A.at<double>(2 * i, 7) = 0;
    A.at<double>(2 * i, 8) = -X * xprime;
    A.at<double>(2 * i, 9) = -Y * xprime;
    A.at<double>(2 * i, 10) = -Z * xprime;

    L.at<double>(2 * i, 0) = xprime;

    // 두 번째 식:
    A.at<double>(2 * i + 1, 0) = 0;
    A.at<double>(2 * i + 1, 1) = 0;
    A.at<double>(2 * i + 1, 2) = 0;
    A.at<double>(2 * i + 1, 3) = 0;
    A.at<double>(2 * i + 1, 4) = X;
    A.at<double>(2 * i + 1, 5) = Y;
    A.at<double>(2 * i + 1, 6) = Z;
    A.at<double>(2 * i + 1, 7) = 1;
    A.at<double>(2 * i + 1, 8) = -X * yprime;
    A.at<double>(2 * i + 1, 9) = -Y * yprime;
    A.at<double>(2 * i + 1, 10) = -Z * yprime;

    L.at<double>(2 * i + 1, 0) = yprime;

```

```

    }

    // 최소자승법으로 DLT 파라미터 추정
    cv::Mat_<double> X;
    cv::solve(A, L, X, cv::DECOMP_QR);

    return X; // 11x1 DLT 파라미터 벡터 반환
}

```

5. 3번과 4번 방법의 정확도를 비교하시오 *단, 정확도는 픽셀단위로 산출하시오 (4번: 변환된 x,y를 다시 c,r로 변환)

-DLT 기반 투영 모델의 정확도를 평가하기 위해, 모델점(GCP)과 검사점(Check Point)에 대해 예측 좌표와 실제 좌표 간의 오차를 계산하고, 각 GCP의 3차원 좌표(XYZ)를 기반으로 두 가지 방식의 투영을 수행

(1) 영상좌표로 바로 변환하는 DLT 모델을 추정하는 방식 (X,Y,Z) to (c,r)

(2) GCP의 c,r좌표를 카메라 좌표계로 변환하여 (X,Y,Z) to (x,y)로 변환시키는 DLT 모델을 추정하는 방식

-각 방식에 대해 예측된 CR 좌표와 실제 GCP의 Truth CR 좌표 간의 차이를 계산하고, 컬럼 방향 오차(dC), 로우 방향 오차(dR), 2D 오차($\sqrt{dC^2 + dR^2}$)를 각각 구하여 RMSE(Root Mean Square Error)로 표현

-이 과정을 모델점과 검사점 각각에 대해 반복 수행하였으며, RMSE 값을 기준으로 모델의 정확도 및 일반화 성능을 비교·분석

[ComputeAndPrintRMSE function]

- 추정된 변환 행렬과 대응점 좌표를 입력받아, 변환된 점과 실제 점 간의 위치 차이를 계산하고, 컬럼(col) 방향, 로우(row) 방향, 전체 2차원 위치 오차에 대한 RMSE(Root Mean Square Error)를 출력하는 함수.

- 모델점(GCP)과 검사점(Check Point)에 대해 각각 두 가지 방식의 정확도를 비교함.

```

void ComputeAndPrintRMSE(const vector<GCP>& model_gcps, const vector<GCP>&
check_gcps,
    const cv::Mat& dltParam1,           // DLT(XYZ → CR) 직접 투영 파라미터 (3x4)
    const cv::Mat& dltParam2,           // DLT(XYZ → xy → CR) 간접 투영 파라미터
    (11x1)
    const IOP& iop)                     // 내부표정요소 (Principal point, cell size 등)
{
    // 내부 람다 함수 정의: GCP 리스트에 대해 RMSE 계산 및 출력
    auto computeRMSE = [&](const vector<GCP>& gcps, const string& label, bool
use_xy_model) {
        // RMSE 출력 헤더
        cout << "\n===== [" << label << "] 모델 예측 좌표 비교 =====" <<

```

```

endl << endl;

cout << left << setw(5) << "No"
    << setw(10) << "GT_c"           // 실제 Col
    << setw(12) << "Pred_c"         // 예측 Col
    << setw(10) << "GT_r"           // 실제 Row
    << setw(12) << "Pred_r"         // 예측 Row
    << setw(8) << "dC"              // Col 오차
    << setw(8) << "dR" << endl; // Row 오차
cout << string(65, '-') << endl;

double sum_c = 0, sum_r = 0, sum_total = 0; // RMSE 누적 변수
int N = gcps.size();

for (int i = 0; i < N; ++i) {
    const auto& gcp = gcps[i];
    cv::Point2d pred;

    // 변환 방식 선택
    // use_xy_model == true: XYZ → xy → CR
    // use_xy_model == false: XYZ → CR 직접
    if (use_xy_model)
        pred = ProjectDLT_XYZtoCR(dltParam2, gcp.xyz, iop);
    else
        pred = ProjectDLT_XYZtoCR_Pixel(dltParam1, gcp.xyz);

    // 실제 좌표 (Ground Truth)
    double gt_c = gcp.order[0].CR.Col;
    double gt_r = gcp.order[0].CR.Row;

    // 예측 오차 계산
    double dc = pred.x - gt_c;
    double dr = pred.y - gt_r;

    // 제곱 오차 누적 (RMSE 계산용)
    sum_c += dc * dc;
    sum_r += dr * dr;
    sum_total += dc * dc + dr * dr;

    // GCP별 예측 결과 출력
    cout << left << setw(5) << i + 1
        << setw(10) << static_cast<int>(gt_c)
        << setw(12) << fixed << setprecision(2) << pred.x
        << setw(10) << static_cast<int>(gt_r)
        << setw(12) << pred.y
        << setw(8) << showpos << dc

```

```

        << setw(8) << dr << noshowpos << endl;
    }

    // RMSE 결과 출력
    cout << fixed << setprecision(8);
    cout << "\nc RMSE : " << sqrt(sum_c / N) << " px\n";
    cout << "r RMSE : " << sqrt(sum_r / N) << " px\n";
    cout << "2D RMSE: " << sqrt(sum_total / N) << " px\n";
};

// 모델점에 대한 두 방식 비교
computeRMSE(model_gcps, "DLT (XYZ→CR) [모델점]", false);
computeRMSE(model_gcps, "DLT (XYZ→xy→CR) [모델점]", true);

// 검사점에 대한 두 방식 비교
computeRMSE(check_gcps, "DLT (XYZ→CR) [검사점]", false);
computeRMSE(check_gcps, "DLT (XYZ→xy→CR) [검사점]", true);
}

```

[ProjectDLT_XYZtoCR]

```

cv::Point2d ProjectDLT_XYZtoCR(const cv::Mat& dltParam, const XYZ& xyz, const IOP&
iop) {
    // (1) 동차 좌표계 분모 계산: W는 3D 점 xyz의 투영 좌표를 정규화하는 데 쓰임
    double W = dltParam.at<double>(8) * xyz.X +
        dltParam.at<double>(9) * xyz.Y +
        dltParam.at<double>(10) * xyz.Z + 1.0; // DLT에서 마지막 항은 보통 1로 설정

    // (2) 사진좌표계(mm) 상의 투영 x', y' 계산 (정규화된 좌표)
    double x_prime = (dltParam.at<double>(0) * xyz.X +
        dltParam.at<double>(1) * xyz.Y +
        dltParam.at<double>(2) * xyz.Z +
        dltParam.at<double>(3)) / W;

    double y_prime = (dltParam.at<double>(4) * xyz.X +
        dltParam.at<double>(5) * xyz.Y +
        dltParam.at<double>(6) * xyz.Z +
        dltParam.at<double>(7)) / W;

    // (3) 픽셀 좌표계 변환을 위한 내부표정 요소 활용
    double sx = iop.CellSize; // 픽셀 당 크기(mm)
    double cx = iop.ColSize / 2.0; // 중심점 x (픽셀 단위)
    double cy = iop.RowSize / 2.0; // 중심점 y (픽셀 단위)
    // (4) 픽셀 좌표계로 변환
    double c = x_prime / sx + cx; // x축 방향: mm → pixel + 중심 이동
    double r = -y_prime / sx + cy; // y축 방향은 부호 반전됨

    return cv::Point2d(c, r); // 픽셀 단위 좌표 반환
}

```

[ProjectDLT_XYZtoCR_Pixel]

```

cv::Point2d ProjectDLT_XYZtoCR_Pixel(const cv::Mat& DLT, const XYZ& xyz) {

    double X = (xyz.X); // * scale_XYZ; // 정규화 스케일 생략
    double Y = (xyz.Y); // * scale_XYZ;
    double Z = (xyz.Z); // * scale_XYZ;

    // (2) 동차 좌표계로 표현 (X, Y, Z, 1)
    cv::Mat_<double> pt(4, 1);
    pt << X, Y, Z, 1.0;

    // (3) DLT 행렬과 곱해서 이미지 평면 상 투영 좌표 계산
    cv::Mat_<double> projected = DLT * pt;

    // (4) 픽셀 좌표로 정규화 (정규화 상수 scale_CR은 주석처리)
    double c = (projected(0, 0) / projected(2, 0)); // / scale_CR;
    double r = (projected(1, 0) / projected(2, 0)); // / scale_CR;

    return cv::Point2d(c, r); // 정규화 후 픽셀 위치 반환
}

```

[XYZ -> CR 에 대한 모델정확도와 검사점정확도 결과]

===== [DLT (XYZ→CR) [모델점]] 모델 예측 좌표 비교 =====						
No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	3391	3390.93	2018	2017.83	-0.07	-0.17
2	3479	3479.11	595	595.11	+0.11	+0.11
3	721	720.78	2816	2816.42	-0.22	+0.42
4	304	304.21	2567	2566.67	+0.21	-0.33
5	887	887.09	2170	2169.96	+0.09	-0.04
6	836	835.88	474	474.01	-0.12	+0.01
c RMSE : 0.14697615 px						
r RMSE : 0.23423064 px						
2D RMSE: 0.27652483 px						
===== [DLT (XYZ→CR) [검사점]] 모델 예측 좌표 비교 =====						
No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	946	942.34	20	20.04	-3.66	+0.04
2	2426	2430.09	796	794.32	+4.09	-1.68
3	1643	1649.01	903	906.24	+6.01	+3.24
4	1654	1662.54	1433	1435.95	+8.54	+2.95
5	2058	2063.41	2219	2224.50	+5.41	+5.50
6	2385	2394.37	2886	2902.83	+9.37	+16.83
c RMSE : 6.53686781 px						
r RMSE : 7.47980372 px						
2D RMSE: 9.93368534 px						

[XYZ -> XY 에 대한 모델정확도와 검사점정확도 결과]

===== [DLT (XYZ→xy→CR) [모델점]] 모델 예측 좌표 비교 =====						
No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	3391	3390.93	2018	2017.83	-0.07	-0.17
2	3479	3479.11	595	595.11	+0.11	+0.11
3	721	720.78	2816	2816.42	-0.22	+0.42
4	304	304.21	2567	2566.67	+0.21	-0.33
5	887	887.09	2170	2169.96	+0.09	-0.04
6	836	835.88	474	474.01	-0.12	+0.01
c RMSE : 0.14697616 px						
r RMSE : 0.23423064 px						
2D RMSE: 0.27652483 px						
===== [DLT (XYZ→xy→CR) [검사점]] 모델 예측 좌표 비교 =====						
No	GT_c	Pred_c	GT_r	Pred_r	dC	dR
1	946	942.34	20	20.04	-3.66	+0.04
2	2426	2430.09	796	794.32	+4.09	-1.68
3	1643	1649.01	903	906.24	+6.01	+3.24
4	1654	1662.54	1433	1435.95	+8.54	+2.95
5	2058	2063.41	2219	2224.50	+5.41	+5.50
6	2385	2394.37	2886	2902.83	+9.37	+16.83
c RMSE : 6.53686796 px						
r RMSE : 7.47980374 px						
2D RMSE: 9.93368545 px						

6. 1 번부터 5 번 과정을 세개의 영상에서 반복수행, 정확도 비교 결과를 자유롭게 고찰하시오

[반복수행]

1. Image Name : R0047634.JPG
2. Number of GCPs : 8
===== [DLT (XYZ→CR) [검사점]] 모델 예측 좌표 비교 =====
c RMSE : 14.10296440 px
r RMSE : 23.92631727 px
2D RMSE: 27.77340928 px
===== [DLT (XYZ→xy→CR) [검사점]] 모델 예측 좌표 비교 =====
c RMSE : 14.10296440 px
r RMSE : 23.92631738 px

2D RMSE: 27.77340937 px

1. Image Name : R0047635.JPG

2. Number of GCPs : 7

===== [DLT (XYZ→CR) [검사점]] 모델 예측 좌표 비교 =====

c RMSE : 5.58036115 px

r RMSE : 0.15802532 px

2D RMSE: 5.58259819 px

===== [DLT (XYZ→xy→CR) [검사점]] 모델 예측 좌표 비교 =====

c RMSE : 5.58036113 px

r RMSE : 0.15802532 px

2D RMSE: 5.58259817 px

1. Image Name : R0047642.JPG

2. Number of GCPs : 9

===== [DLT (XYZ→CR) [검사점]] 모델 예측 좌표 비교 =====

c RMSE : 31.93990667 px

r RMSE : 17.13638624 px

2D RMSE: 36.24656358 px

===== [DLT (XYZ→xy→CR) [검사점]] 모델 예측 좌표 비교 =====

c RMSE : 31.93990661 px

r RMSE : 17.13638619 px

2D RMSE: 36.24656350 px

- 여러 개의 영상에 대해 DLT 기반 투영 모델을 적용하고, (1) XYZ 에서 직접 영상좌표(CR)로 변환한 모델과 (2) XYZ 를 xy 로 투영 후 내부표정(IOP)을 이용해 CR 로 변환하는 두 가지 방식을 비교하였다.

- 실험 결과, 두 방법 모두 RMSE 기준으로 매우 유사한 정확도를 보였으며, 두 모델의 차이는 아주 미세했다.

- 이러한 결과는 두 방식 모두 동일한 DLT 기반 투영 모델을 사용하고 있으며, 내부표정요소(IOP)가 정확하게 적용되었을 경우, XY 평면으로의 중간 변환이 전체 정확도에 큰 영향을 미치지 않는다는 점을 나타낸다고 보았다. 특히 GCP 분포가 균일하고 이미지 왜곡이 상대적으로 작을 경우, 두 방식 모두 안정적인 예측을 수행할 수 있었으며, 실질적인 성능 차이는 미미하였다.

- 따라서 모델 선택은 정확도보다는 구현의 간편성, 계산 속도, 또는 특정 응용 상황에서의 요구사항 등을 기준으로 결정하는 것이 보다 합리적일 수 있다고 생각했다.

5. Part3 Precision Sensor Model Solution

1. 정밀센서모델 가이드를 통해 FrameCamera(h, cpp)파일 내 정밀센서모델 코드를 작성하시오

[FrameCamera.h]

```
EOP PrecisionSensorModel(std::vector<GCP>& gcps, const EOP& initialEop, const IOP& iop);  
cv::Mat_<double> Get_dR_dOmega(double omega, double phi, double kappa);  
cv::Mat_<double> Get_dR_dPhi(double omega, double phi, double kappa);  
cv::Mat_<double> Get_dR_dKappa(double omega, double phi, double kappa);  
cv::Mat_<double> Get_dRvec(const cv::Mat_<double>& dR, double dX, double dY, double dZ);
```

[FrameCamera.cpp - PrecisionSensorModel]

```
EOP PrecisionSensorModel(vector<GCP>& gcps, const EOP& initialEop, const IOP& iop) {  
    EOP eop = initialEop; // 초기 EOP 복사  
    const double tolerance = 1e-6; // 수렴 조건  
    bool converged = false;  
  
    while (!converged) {  
  
        // 1. 현재 EOP를 이용해 회전행렬 계산  
        cv::Mat_<double> R = GetRotationMatrix(eop.O * ToRad, eop.P * ToRad, eop.K * ToRad);  
  
        // 1-1. 회전 미분 행렬 계산 (Group C)  
        cv::Mat_<double> dR_Omega = Get_dR_dOmega(eop.O * ToRad, eop.P * ToRad, eop.K * ToRad);  
        cv::Mat_<double> dR_Phi = Get_dR_dPhi(eop.O * ToRad, eop.P * ToRad, eop.K * ToRad);  
        cv::Mat_<double> dR_Kappa = Get_dR_dKappa(eop.O * ToRad, eop.P * ToRad, eop.K * ToRad);  
  
        // 회전 성분 벡터의 미분 결과 계산 함수  
  
        int m = gcps.size();  
        cv::Mat_<double> A(2 * m, 6, 0.0); // 관측 방정식 계수 행렬 (2m x 6)  
        cv::Mat_<double> L(2 * m, 1, 0.0); // 관측 오차 벡터 (2m x 1)  
  
        // 2. 모든 GCP에 대해 관측 방정식 구성  
        for (int i = 0; i < m; ++i) {  
            GCP& gcp = gcps[i];  
            XYZ ground = gcp.xyz;  
            ColRow cr = gcp.order[0].CR;  
  
            double dX = ground.X - eop.Xs;  
            double dY = ground.Y - eop.Ys;  
            double dZ = ground.Z - eop.Zs;
```

```

double x = (cr.Col - iop.ColSize / 2.0) * iop.CellSize;
double y = -(cr.Row - iop.RowSize / 2.0) * iop.CellSize;

double out_c, out_r;
InverseMapping(iop, eop, R, ground.X, ground.Y, ground.Z, out_c, out_r);
double x_mapped = (out_c - iop.ColSize / 2.0) * iop.CellSize;
double y_mapped = -(out_r - iop.RowSize / 2.0) * iop.CellSize;

// Group C: Rxx, Ryy, Rzz 계산
double Rxx = R(0, 0) * dX + R(0, 1) * dY + R(0, 2) * dZ;
double Ryy = R(1, 0) * dX + R(1, 1) * dY + R(1, 2) * dZ;
double Rzz = R(2, 0) * dX + R(2, 1) * dY + R(2, 2) * dZ;

// 위치(Xs, Ys, Zs)에 대한 편미분
double dFx_dXs = iop.Focal * (R(0, 0) * Rzz - Rxx * R(2, 0)) / (Rzz * Rzz);
double dFx_dYs = iop.Focal * (R(0, 1) * Rzz - Rxx * R(2, 1)) / (Rzz * Rzz);
double dFx_dZs = iop.Focal * (R(0, 2) * Rzz - Rxx * R(2, 2)) / (Rzz * Rzz);

double dFy_dXs = iop.Focal * (R(1, 0) * Rzz - Ryy * R(2, 0)) / (Rzz * Rzz);
double dFy_dYs = iop.Focal * (R(1, 1) * Rzz - Ryy * R(2, 1)) / (Rzz * Rzz);
double dFy_dZs = iop.Focal * (R(1, 2) * Rzz - Ryy * R(2, 2)) / (Rzz * Rzz);

// 회전 성분에 대한 편미분 (Mat 버전)
cv::Mat_<double> dRvec_Omega = Get_dRvec(dR_Omega, dX, dY, dZ);
cv::Mat_<double> dRvec_Phi = Get_dRvec(dR_Phi, dX, dY, dZ);
cv::Mat_<double> dRvec_Kappa = Get_dRvec(dR_Kappa, dX, dY, dZ);

double dFx_dO = -iop.Focal * (dRvec_Omega(0, 0) * Rzz - Rxx *
dRvec_Omega(2, 0)) / (Rzz * Rzz);
double dFx_dP = -iop.Focal * (dRvec_Phi(0, 0) * Rzz - Rxx * dRvec_Phi(2, 0))
/ (Rzz * Rzz);
double dFx_dK = -iop.Focal * (dRvec_Kappa(0, 0) * Rzz - Rxx *
dRvec_Kappa(2, 0)) / (Rzz * Rzz);

double dFy_dO = -iop.Focal * (dRvec_Omega(1, 0) * Rzz - Ryy *
dRvec_Omega(2, 0)) / (Rzz * Rzz);
double dFy_dP = -iop.Focal * (dRvec_Phi(1, 0) * Rzz - Ryy * dRvec_Phi(2, 0))
/ (Rzz * Rzz);
double dFy_dK = -iop.Focal * (dRvec_Kappa(1, 0) * Rzz - Ryy *
dRvec_Kappa(2, 0)) / (Rzz * Rzz);

// A 행렬 채우기
A(2 * i, 0) = dFx_dXs; A(2 * i, 1) = dFx_dYs; A(2 * i, 2) = dFx_dZs;
A(2 * i, 3) = dFx_dO; A(2 * i, 4) = dFx_dP; A(2 * i, 5) = dFx_dK;
A(2 * i + 1, 0) = dFy_dXs; A(2 * i + 1, 1) = dFy_dYs; A(2 * i + 1, 2) = dFy_dZs;
A(2 * i + 1, 3) = dFy_dO; A(2 * i + 1, 4) = dFy_dP; A(2 * i + 1, 5) = dFy_dK;

// L 벡터 채우기
L(2 * i, 0) = x - x_mapped;
L(2 * i + 1, 0) = y - y_mapped;

```

```

    }

    // 3. 최소자승 해 계산
    cv::Mat_<double> At = A.t();
    cv::Mat_<double> ATA = At * A;
    cv::Mat_<double> ATL = At * L;
    cv::Mat_<double> X;

    // SVD로 안전하게 풀기
    cv::solve(ATA, ATL, X, cv::DECOMP_SVD);

    // 4. EOP 업데이트
    eop.Xs += X(0, 0);
    eop.Ys += X(1, 0);
    eop.Zs += X(2, 0);
    eop.O += X(3, 0);
    eop.P += X(4, 0);
    eop.K += X(5, 0);

    // 5. 수렴 검사
    if (cv::norm(X) < tolerance) converged = true;
}

return eop; // 최종 결과 리턴
}

```

Step 1 목표설정

$$x = -f \cdot \frac{r_{11}(X - X_s) + r_{12}(Y - Y_s) + r_{13}(Z - Z_s)}{r_{31}(X - X_s) + r_{32}(Y - Y_s) + r_{33}(Z - Z_s)}$$

$$y = -f \cdot \frac{r_{21}(X - X_s) + r_{22}(Y - Y_s) + r_{23}(Z - Z_s)}{r_{31}(X - X_s) + r_{32}(Y - Y_s) + r_{33}(Z - Z_s)}$$

목표: 공선조건식을 통해 외부표정요소 $EOP = \{X_s, Y_s, Z_s, \Omega, \Phi, K\}$ 변수를 추정하는 것

Step 2 공선조건식 선형화

$$x = -f \cdot \frac{r_{11}(X - X_s) + r_{12}(Y - Y_s) + r_{13}(Z - Z_s)}{r_{31}(X - X_s) + r_{32}(Y - Y_s) + r_{33}(Z - Z_s)}$$

$$y = -f \cdot \frac{r_{21}(X - X_s) + r_{22}(Y - Y_s) + r_{23}(Z - Z_s)}{r_{31}(X - X_s) + r_{32}(Y - Y_s) + r_{33}(Z - Z_s)}$$

-테일러 전개를 통한 선형화 결과

$$\begin{aligned} \Delta x &= (\partial f x / \partial X_s) \Delta X_s + (\partial f x / \partial Y_s) \Delta Y_s + (\partial f x / \partial Z_s) \Delta Z_s + (\partial f x / \partial \omega) \Delta \omega + (\partial f x / \partial \varphi) \Delta \varphi + (\partial f x / \partial \kappa) \Delta \kappa \\ \Delta y &= (\partial f y / \partial X_s) \Delta X_s + (\partial f y / \partial Y_s) \Delta Y_s + (\partial f y / \partial Z_s) \Delta Z_s + (\partial f y / \partial \omega) \Delta \omega + (\partial f y / \partial \varphi) \Delta \varphi + (\partial f y / \partial \kappa) \Delta \kappa \end{aligned}$$

Step 2-1 선형화를 위한 미분함수 작성

[Group C 미분함수 작성]

손으로 RotationMatrix 에 대해 각 omega phi kappa 에 대해 편미분 한 값을 함수로 지정해 omega phi kappa 를 입력받으면 Group C 를 계산

```
cv::Mat_<double> Get_dR_dOmega(double omega, double phi, double kappa) {
    double cw = cos(omega);
    double sw = sin(omega);
    double cp = cos(phi);
    double sp = sin(phi);
    double ck = cos(kappa);
    double sk = sin(kappa);

    cv::Mat_<double> dR(3, 3);

    dR(0, 0) = 0.0;
    dR(0, 1) = cw * sp * ck - sw * sk;
    dR(0, 2) = sw * sp * ck + cw * sk;

    dR(1, 0) = 0.0;
    dR(1, 1) = -cw * sp * sk - sw * ck;
    dR(1, 2) = -sw * sp * sk + cw * ck;

    dR(2, 0) = 0.0;
    dR(2, 1) = -cw * cp;
    dR(2, 2) = -sw * cp;

    return dR;
}

cv::Mat_<double> Get_dR_dPhi(double omega, double phi, double kappa) {
    double cw = cos(omega);
    double sw = sin(omega);
    double cp = cos(phi);
    double sp = sin(phi);
    double ck = cos(kappa);
    double sk = sin(kappa);

    cv::Mat_<double> dR(3, 3);

    dR(0, 0) = -sp * ck;
    dR(0, 1) = sw * cp * ck;
    dR(0, 2) = -cw * cp * ck;

    dR(1, 0) = sp * sk;
    dR(1, 1) = -sw * cp * sk;
    dR(1, 2) = cw * cp * sk;

    dR(2, 0) = cp;
```

```

    dR(2, 1) = sw * sp;
    dR(2, 2) = -cw * sp;

    return dR;
}
cv::Mat_<double> Get_dR_dKappa(double omega, double phi, double kappa) {
    double cw = cos(omega);
    double sw = sin(omega);
    double cp = cos(phi);
    double sp = sin(phi);
    double ck = cos(kappa);
    double sk = sin(kappa);

    cv::Mat_<double> dR(3, 3);

    dR(0, 0) = -cp * sk;
    dR(0, 1) = -sw * sp * sk + cw * ck;
    dR(0, 2) = cw * sp * sk + sw * ck;

    dR(1, 0) = -cp * ck;
    dR(1, 1) = -sw * sp * ck - cw * sk;
    dR(1, 2) = cw * sp * ck - sw * sk;

    dR(2, 0) = 0.0;
    dR(2, 1) = 0.0;
    dR(2, 2) = 0.0;

    return dR;
}

```

[Rxx, Ryy, Rzz 작성]

```

double Rxx = R(0, 0) * dX + R(0, 1) * dY + R(0, 2) * dZ;
double Ryy = R(1, 0) * dX + R(1, 1) * dY + R(1, 2) * dZ;
double Rzz = R(2, 0) * dX + R(2, 1) * dY + R(2, 2) * dZ;

```

[Group A, B 작성]

```
cv::Mat_<double> R = GetRotationMatrix(eop.O * ToRad, eop.P * ToRad, eop.K * ToRad);
```

```

double dFx_dXs = iop.Focal * (R(0, 0) * Rzz - Rxx * R(2, 0)) / (Rzz * Rzz);
double dFx_dYs = iop.Focal * (R(0, 1) * Rzz - Rxx * R(2, 1)) / (Rzz * Rzz);
double dFx_dZs = iop.Focal * (R(0, 2) * Rzz - Rxx * R(2, 2)) / (Rzz * Rzz);

double dFy_dXs = iop.Focal * (R(1, 0) * Rzz - Ryy * R(2, 0)) / (Rzz * Rzz);
double dFy_dYs = iop.Focal * (R(1, 1) * Rzz - Ryy * R(2, 1)) / (Rzz * Rzz);
double dFy_dZs = iop.Focal * (R(1, 2) * Rzz - Ryy * R(2, 2)) / (Rzz * Rzz);

// 회전 성분에 대한 편미분 (Mat 버전)
cv::Mat_<double> dRvec_Omega = Get_dRvec(dR_Omega, dX, dY, dZ);
cv::Mat_<double> dRvec_Phi = Get_dRvec(dR_Phi, dX, dY, dZ);
cv::Mat_<double> dRvec_Kappa = Get_dRvec(dR_Kappa, dX, dY, dZ);

```

```

double dFx_dO = -iop.Focal * (dRvec_Omega(0, 0) * Rzz - Rxx * dRvec_Omega(2, 0)) /
(Rzz * Rzz);
double dFx_dP = -iop.Focal * (dRvec_Phi(0, 0) * Rzz - Rxx * dRvec_Phi(2, 0)) / (Rzz * Rzz);
double dFx_dK = -iop.Focal * (dRvec_Kappa(0, 0) * Rzz - Rxx * dRvec_Kappa(2, 0)) / (Rzz *
Rzz);

double dFy_dO = -iop.Focal * (dRvec_Omega(1, 0) * Rzz - Ryy * dRvec_Omega(2, 0)) /
(Rzz * Rzz);
double dFy_dP = -iop.Focal * (dRvec_Phi(1, 0) * Rzz - Ryy * dRvec_Phi(2, 0)) / (Rzz * Rzz);
double dFy_dK = -iop.Focal * (dRvec_Kappa(1, 0) * Rzz - Ryy * dRvec_Kappa(2, 0)) / (Rzz *
Rzz);

```

Step 3 AX = L 만들기

- 각 Ground Control Point(GCP)에 대해 실제 관측 영상과 eop 기반 InverseMapping 으로 투영된 좌표 간의 차이를 계산하여, 오차 벡터 L 에 저장
- 이 오차는 공선 조건식 기반 투영과 실제 값 간의 불일치를 의미하며, 잔차로 해석
- 관측 방정식의 계수행렬 A 와 오차벡터 L 을 이용하여 최소제곱법을 수행

```

double out_c, out_r;
InverseMapping(iop, eop, R, ground.X, ground.Y, ground.Z, out_c, out_r);
double x_mapped = (out_c - iop.ColSize / 2.0) * iop.CellSize;

double y_mapped = -(out_r - iop.RowSize / 2.0) * iop.CellSize;
// A 행렬 채우기
A(2 * i, 0) = dFx_dXs; A(2 * i, 1) = dFx_dYs; A(2 * i, 2) = dFx_dZs;
A(2 * i, 3) = dFx_dO; A(2 * i, 4) = dFx_dP; A(2 * i, 5) = dFx_dK;
A(2 * i + 1, 0) = dFy_dXs; A(2 * i + 1, 1) = dFy_dYs; A(2 * i + 1, 2) = dFy_dZs;
A(2 * i + 1, 3) = dFy_dO; A(2 * i + 1, 4) = dFy_dP; A(2 * i + 1, 5) = dFy_dK;

// L 벡터 채우기
L(2 * i, 0) = x - x_mapped;
L(2 * i + 1, 0) = y - y_mapped;

```

Step 4 반복적으로 최소제곱법 수행

- 선형시스템을 안정적으로 풀기 위해 SVD 기반의 cv::solve()함수를 사용
- 계산된 결과 벡터 X 는 카메라 위치(Xs, Ys, Zs)와 omega, phi, kappa 에 대한 보정량
- 보정량을 기존의 EOP 에 누적하여 새로운 추정값으로 업데이트
- 보정량 벡터 X 의 노름(norm)이 수렴 기준인 1 * e-06 보다 작을 경우 수렴한 것으로 판단하고 반복을 종료

```

EOP PrecisionSensorModel(vector<GCP>& gcps, const EOP& initialEop, const IOP& iop) {
    EOP eop = initialEop; // 초기 EOP 복사
    const double tolerance = 1e-6; // 수렴 조건

```

```

bool converged = false;

while (!converged) {
    ///~~~~관측방정식 구성~~~~///
    // 3. 최소자승 해 계산
    cv::Mat_<double> At = A.t();
    cv::Mat_<double> ATA = At * A;
    cv::Mat_<double> ATL = At * L;
    cv::Mat_<double> X;

    // SVD로 안전하게 풀기
    cv::solve(ATA, ATL, X, cv::DECOMP_SVD);

    // 4. EOP 업데이트
    eop.Xs += X(0, 0);
    eop.Ys += X(1, 0);
    eop.Zs += X(2, 0);
    eop.O += X(3, 0);
    eop.P += X(4, 0);
    eop.K += X(5, 0);

    // 5. 수렴 검사
    if (cv::norm(X) < tolerance) converged = true;
}

return eop; // 최종 결과 리턴
}

```

2. 최소 6 개 이상의 GCP 를 가지는 영상 하나를 선정하시오

-12 개의 GCP 를 가지는 R0047633.JPG 로 영상 선정.

3. 그 중 5 개의 GCP 정보를 추출하여 모델링에 사용하고 나머지 점은 검사점으로 사용하시오

-원하는 GCP번호를 선정해 모델점과 검사점으로 분류하였다.



가장 5 개 GCP 점의 분포가 넓게 퍼지도록 1, 2, 5, 6, 8 GCP 를 선정

[maincode]

```
vector<int> selectedModelIndices = { 1, 2, 5, 6, 8 }; // 원하는 GCP 번호들

for (int i = 0; i < ImageGCP.size(); ++i) {
    // GCP 번호는 1-based로 생각하고 있으므로 (i + 1)
    if (find(selectedModelIndices.begin(), selectedModelIndices.end(), i + 1) !=
        selectedModelIndices.end()) {
        ModelGCP.push_back(ImageGCP[i]); // 선택된 모델점
    }
    else {
        CheckGCP.push_back(ImageGCP[i]); // 나머지 검사점
    }
}
```

4. 모델점과 검사점의 초기 Mapping 오차를 구하시오

(X,Y,Z to c,r : InverseMapping)

-오차를 정량적으로 구하기 위해 GCP(Ground Control Points)의 실제 좌표와 EOP(IOP 포함) 기반 InverseMapping 영상좌표(CR) 간의 차이를 계산하여 RMSE(Root Mean Square Error), 평균 오차를 구하는 함수를 작성하였다.

```
double ComputeRMSE(const std::vector<GCP>& gcps, const EOP& eop, const IOP& iop) {
    double sumSquaredError = 0.0; // 제곱합 누적값
    double sumError = 0.0;        // 오차 누적값
    std::vector<double> errors;    // 개별 GCP 오차 저장용

    int count = 0;

    // EOP에서 회전행렬 계산 (O, P, K는 degree → radian 변환 필요)
    cv::Mat_<double> R = GetRotationMatrix(eop.O * ToRad, eop.P * ToRad, eop.K *
        ToRad);

    for (const auto& gcp : gcps) {
        // GCP의 3차원 위치
        double X = gcp.xyz.X;
        double Y = gcp.xyz.Y;
        double Z = gcp.xyz.Z;

        // 실제 영상좌표 (Ground Truth)
        double c_true = gcp.order[0].CR.Col;
        double r_true = gcp.order[0].CR.Row;

        double c_calc, r_calc;

        // Inverse collinearity 식으로 예측된 영상좌표 계산
    }
```

```

InverseMapping(iop, eop, R, X, Y, Z, c_calc, r_calc);

// 예측좌표와 실제좌표 간의 제공오차 및 오차
double squaredError = std::pow(c_calc - c_true, 2) + std::pow(r_calc - r_true, 2);
double error = std::sqrt(squaredError);

sumSquaredError += squaredError;
sumError += error;
errors.push_back(error);

// 개별 GCP 오차 출력
cout << fixed << setprecision(2);
cout << count + 1 << "번 GCP Error : " << error << " px" << endl;

count++;
}

// 평균 오차 및 RMSE 계산
double averageError = sumError / count;
double rmse = std::sqrt(sumSquaredError / count);
// 결과 출력
cout << "\nAverage Error      : " << fixed << setprecision(2) << averageError << " px"
<< endl;

return rmse;
}

```

[초기오차 결과]

```

===== [ Initial Mapping Evaluation ] =====
----- [Control Points Mapping Error] -----
1번 GCP Error : 32.15 px
2번 GCP Error : 33.70 px
3번 GCP Error : 30.01 px
4번 GCP Error : 17.87 px
5번 GCP Error : 32.20 px

Average Error      : 29.18 px
----- [Check Points Mapping Error] -----
1번 GCP Error : 38.47 px
2번 GCP Error : 39.03 px
3번 GCP Error : 12.86 px
4번 GCP Error : 35.35 px
5번 GCP Error : 36.80 px
6번 GCP Error : 32.03 px
7번 GCP Error : 35.45 px

Average Error      : 32.86 px
----- [ Mapping RMSE ] -----
Control Points RMSE (Before) : 29.75 px
Check Points RMSE (Before) : 33.92 px

```

5. 5 개의 GCP 정보를 통해 해당 영상의 정밀센서모델링을 수행하시오

-PrecisionSensorModel 함수에 model gcp, eop, iop 를 인자로 정밀한 eop 를 얻는다

```
EOP_CorrectedEOP = PrecisionSensorModel(ModelGCP, ImageEOP, ImageIOP);
```

6. 센서모델링 이후 보정된 EOP 를 통해 Mapping 오차를 구하시오 (X,Y,Z to c,r : InverseMapping)

-InverseMapping 을 사용한 ComputeRMSE 함수를 활용해 평균 오차와 RMSE 를 구한다.

```
cout << "----- [Control Points Mapping Error] -----\\n" << endl;
double RMSE_Model_After = ComputeRMSE(ModelGCP, CorrectedEOP, ImageIOP);
cout << "\\n----- [Check Points Mapping Error] -----\\n" << endl;
double RMSE_Check_After = ComputeRMSE(CheckGCP, CorrectedEOP, ImageIOP);
```

[센서모델링 이후 결과]

```
===== [ Precision Sensor Modeling ] =====
----- [Control Points Mapping Error] -----
1번 GCP Error : 1.91 px
2번 GCP Error : 2.79 px
3번 GCP Error : 0.61 px
4번 GCP Error : 0.46 px
5번 GCP Error : 4.14 px

Average Error      : 1.98 px

----- [Check Points Mapping Error] -----
1번 GCP Error : 10.83 px
2번 GCP Error : 12.77 px
3번 GCP Error : 2.27 px
4번 GCP Error : 9.97 px
5번 GCP Error : 10.09 px
6번 GCP Error : 3.01 px
7번 GCP Error : 2.85 px

Average Error      : 7.40 px

----- [ Mapping RMSE ] -----
Control Points RMSE (After) : 2.41 px
Check Points RMSE (After) : 8.48 px
```

7. 실험방법/실험결과에 대해 자유롭게 고찰하시오

- Precision Sensor Model 을 적용하기 전, Initial Mapping 단계에서의 Control Points RMSE (Before) : 29.75 px, Check Points RMSE (Before) : 33.92 px 로 나타났다. 그러나 공선 조건식 기반 센서모델링을 수행한 결과, Control Points RMSE 는 Control Points RMSE (After) : 2.41 px, Check Points RMSE (After) : 8.48 px 로 유의미한 오차 감소가 확인되었다. 평균 오차 기준으로 감소가 확인되었다. 이는 공선조건식과 Least Squares 기반의 파라미터 추정이 학습에 사용된 점들을 중심으로 높은 정확도를 달성했음을 보여준다. 특히 외부표정요소 보정을 통해 초기 파라미터 오차가 보정되면서, 모델이 현실 투영 구조에 점차 수렴하는 경향이 뚜렷하게 나타났다.

하지만 1px 수준의 오차도 정밀한 데이터가 필요할 때는 큰 오차로 다가올 수 있다. 그렇기에 오차수준을 줄 일 수 있는 방안을 생각해보았다.

- 1) 모델링에 사용할 GCP 의 개수를 늘린다.
- 2) 모델링에 사용할 GCP 의 분포를 골고루 퍼지도록 한다.



위의 사진을 바탕으로 1) 2)번 조건을 여럿 달리하여 분석해보니, 편향된 GCP 5 개를 선택했을때 20px 이 넘는 RMSE 가 검사점에서 나타날 수 있었고, 분포가 골고루 퍼져있을수록 매우 눈에 띄는 오차감소를 보여주었다. 또한 GCP 의 개수와 검사점 RMSE 가 완전한 반비례는 아니지만 음의 경향성을 띄는 모습을 볼 수 있었다. 이에 따라 적절한 GCP 의 선택이 모델링에 영향을 미침을 알 수 있었다.

6, 7, 8., 9, 10 GCP 모델링 시 Check Points RMSE (After) : 29.31 px

- 운동장 위쪽에 모여있어 RMSE 가 크게 나타난다.

6, 7, 8, 9, 10, 11 GCP 모델링 시 Check Points RMSE (After) : 22.13 px

6, 7, 8, 9, 10, 11, 12 GCP 모델링 시 Check Points RMSE (After) : 9.13 px

- GCP 개수가 늘어날수록 줄어드는 경향성을 띄는 RMSE

Control Points RMSE 는 1-4px 정도의 수치를 보였다.

이는 공선조건식이 실제 환경에서는 다양한 외부 요인에 의해 일반화 성능이 제한될 수 있음을 시사한다. 주요 원인으로는 GCP의 공간 분포 불균형, Check Point의 측정 오차, 광학 왜곡 등이 고려될 수 있다.

그럼에도 불구하고 공선조건식 기반 모델은 단순 투영 모델보다 실제 촬영 환경을 더 정확하게 반영할 수 있어 기술적으로 더 의미 있다. 앞으로는 더 정밀하고 다양한 상황에 대응할 수 있는 카메라 모델링 기법을 제대로 공부해보고 싶다.

[HardCopy] – Midterm_1.cpp, Midterm_2.cpp, Midterm_3.cpp

```
#include "Midterm.h"
#include "FrameCamera.h"
#include "function.h"

void EvaluateRMSE(const cv::Mat_<double>& H, const cv::Mat_<double>& srcModel, const cv::Mat_<double>& dstModel,
const cv::Mat_<double>& srcTest, const cv::Mat_<double>& dstTest);
void EvaluateRMSE(const cv::Mat_<double>& H, const cv::Mat_<double>& srcTest, const cv::Mat_<double>& dstTest);
void PrintMatrixNicely(const cv::Mat_<double>& mat, const string& title);

// Midterm_1: Affine + Homography 수행
void Midterm_1() {

    string folder_dir = "C:/Users/tnddm/OneDrive/바탕 화면/codepaper/opencv/resource/";
    string image_name1 = "pixel_R0047634.txt";
    string image_name2 = "pixel_R0047635.txt";

    // 포인트 읽기
    cv::Mat_<double> A1, B1, A2, B2;
    cv::Mat_<double> A1t, B1t, A2t, B2t;
    readPoints(folder_dir + image_name1, A1, B1, A2, B2);
    readPoints(folder_dir + image_name2, A1t, B1t, A2t, B2t);

    // ----- Affine -----
    cout << "===== "<<endl;
    cout << "===== [ Affine Transformation ]===== Wn";
    cout << "===== "<< endl;

    // Affine up
    auto affineUp = EstimateAffineMatrix(A1, A1t);
    PrintMatrixNicely(affineUp, "Affine Matrix (up)");
    EvaluateRMSE(affineUp, A1, A1t, A2, A2t);

    // Affine down
    auto affineDown = EstimateAffineMatrix(B1, B1t);
    PrintMatrixNicely(affineDown, "Affine Matrix (down)");
    EvaluateRMSE(affineDown, B1, B1t, B2, B2t);

    // Affine updown
    cv::Mat_<double> AB1, AB2, AB1t, AB2t;
    cv::vconcat(A1, B1, AB1);
    cv::vconcat(A1t, B1t, AB1t);
    cv::vconcat(A2, B2, AB2);
    cv::vconcat(A2t, B2t, AB2t);
    auto affineAll = EstimateAffineMatrix(AB1, AB1t);
    PrintMatrixNicely(affineAll, "Affine Matrix (updown)");
    EvaluateRMSE(affineAll, AB1, AB1t, AB2, AB2t);

    cout << "WnWnWnWnWn";

    // ----- Homography -----
    cout << "===== "<< endl;
    cout << "===== [ Homography Transformation ]===== Wn";
```

```

cout << "===== " << endl;

// Homography up
auto homoUp = EstimateHomographyMatrix(A1, A1t);
PrintMatrixNicely(homoUp, "Homography Matrix (up)");
EvaluateRMSE(homoUp, A1, A1t, A2, A2t);

// Homography down
auto homoDown = EstimateHomographyMatrix(B1, B1t);
PrintMatrixNicely(homoDown, "Homography Matrix (down)");
EvaluateRMSE(homoDown, B1, B1t, B2, B2t);

// Homography updown
auto homoAll = EstimateHomographyMatrix(AB1, AB1t);
PrintMatrixNicely(homoAll, "Homography Matrix (updown)");
EvaluateRMSE(homoAll, AB1, AB1t, AB2, AB2t);
}

void EvaluateRMSE(const cv::Mat_<double>& H, const cv::Mat_<double>& srcModel,
const cv::Mat_<double>& dstModel, const cv::Mat_<double>& srcTest,
const cv::Mat_<double>& dstTest) {
cout << "Wn===== Model Point Accuracy =====Wn";
EvaluateRMSE(H, srcModel, dstModel);
cout << "Wn===== Test Point Accuracy =====Wn";
EvaluateRMSE(H, srcTest, dstTest);
}

void EvaluateRMSE(const cv::Mat_<double>& H,
const cv::Mat_<double>& srcTest,
const cv::Mat_<double>& dstTest) {
int N = srcTest.rows;
double sum_x_err = 0, sum_y_err = 0, sum_total = 0;

cout << "Wn===== Model Prediction Comparison =====Wn";
cout << "WnNo  GT_c    Pred_c  GT_r    Pred_r  dC    dRWn";
cout << "-----Wn";

for (int i = 0; i < N; ++i) {
// source point (homogeneous)
cv::Mat_<double> pt(3, 1);
pt << srcTest(i, 0), srcTest(i, 1), 1.0;

// 변환
cv::Mat_<double> pred = H * pt;
double pred_x = pred(0, 0) / pred(2, 0);
double pred_y = pred(1, 0) / pred(2, 0);

double gt_x = dstTest(i, 0);
double gt_y = dstTest(i, 1);

double dx = pred_x - gt_x;
double dy = pred_y - gt_y;

sum_x_err += dx * dx;
sum_y_err += dy * dy;
sum_total += dx * dx + dy * dy;

printf("%-3d %-8.2f %-8.2f %-8.2f %-8.2f %+6.2f %+6.2fWn",
i + 1, gt_x, pred_x, gt_y, pred_y, dx, dy);
}

cout << "WnC-error RMSE : " << sqrt(sum_x_err / N) << " pxWn";
cout << "R-error RMSE : " << sqrt(sum_y_err / N) << " pxWn";
cout << "2D RMSE : " << sqrt(sum_total / N) << " pxWn";
cout << "=====Wn";
}

```

```

void PrintMatrixNicely(const cv::Mat_<double>& mat, const string& title) {
    cout << "Wn[" << title << "]Wn";
    cout << fixed << setprecision(6);
    for (int i = 0; i < mat.rows; ++i) {
        cout << "(";
        for (int j = 0; j < mat.cols; ++j) {
            cout << setw(12) << mat(i, j);
            if (j < mat.cols - 1) cout << ", ";
        }
        cout << ")" << endl;
    }
}

#include "Midterm.h"
#include "FrameCamera.h"
#include "function.h"

void ComputeAndPrintRMSE(const vector<GCP>& model_gcps, const vector<GCP>& check_gcps,
    const cv::Mat& dltParam1, const cv::Mat& dltParam2, const IOP& iop);

```

```

void Midterm_2() {
    // 1. 경로 설정
    string folder_dir = "C:/Users/tnddm/OneDrive/바탕 화면/codepaper/opencv/resource/";
    string image_name = "R0047633.JPG";
    string IOP_name = folder_dir + "IOFile_Metric.txt";
    string EOP_name = folder_dir + "input_EOP.txt";
    string GCP_name = folder_dir + "GCP.txt";

    // 2. 파일 읽기
    IOP ImageIOP;
    vector<EOP> ImageEOPs;
    EOP ImageEOP;
    vector<GCP> ImageGCPs;
    vector<GCP> ImageGCP;

    ReadIOP(IOP_name, ImageIOP);
    ReadEOP(EOP_name, ImageEOPs);
    ReadGCP(GCP_name, ImageGCPs);
    ReadImageEOP(ImageEOPs, image_name, ImageEOP);
    ReadImageGCP(ImageGCPs, image_name, ImageGCP);

    // 3. GCP 정보 출력
    PrintGCPInfo(ImageGCP, image_name);

    // 4. 모델점/검사점 분리 (앞 6개 모델링)
    vector<GCP> ModelGCP;
    vector<GCP> CheckGCP;
    for (int i = 0; i < ImageGCP.size(); ++i) {
        if (i < 6) ModelGCP.push_back(ImageGCP[i]);
        else CheckGCP.push_back(ImageGCP[i]);
    }

    // 5. DLT 파라미터 추정
    cv::Mat DLT_CR_Params = EstimateDLT_Model_XYZtoCR(ModelGCP);
    cv::Mat DLT_XY_Params = EstimateDLT_Model_XYZtoXY(ModelGCP, ImageIOP);

    ComputeAndPrintRMSE(ModelGCP, CheckGCP, DLT_CR_Params, DLT_XY_Params, ImageIOP);
}

// 두 가지 방식의 DLT 기반 투영 결과를 비교하고 RMSE를 출력하는 함수
void ComputeAndPrintRMSE(const vector<GCP>& model_gcps, const vector<GCP>& check_gcps,
    const cv::Mat& dltParam1, const cv::Mat& dltParam2, const IOP& iop) {

    // RMSE 계산 및 출력 함수 (내부 람다식)
    auto computeRMSE = [&](const vector<GCP>& gcps, const string& label, bool use_xy_model) {

```

```

// 결과 헤더 출력
cout << "Wn===== [ " << label << " ] 모델 예측 좌표 비교 =====" << endl << endl;
cout << left << setw(5) << "No"
    << setw(10) << "GT_c"           // Ground Truth col
    << setw(12) << "Pred_c"         // 예측 col
    << setw(10) << "GT_r"           // Ground Truth row
    << setw(12) << "Pred_r"         // 예측 row
    << setw(8) << "dC"              // col 방향 오차
    << setw(8) << "dR" << endl;    // row 방향 오차
cout << string(65, '-') << endl;

double sum_c = 0, sum_r = 0, sum_total = 0; // RMSE 누적 합
int N = gcps.size();

// 각 GCP에 대해 예측 및 오차 계산
for (int i = 0; i < N; ++i) {
    const auto& gcp = gcps[i];
    cv::Point2d pred;

    // use_xy_model이 true면 xy 중간단계를 거치는 투영 함수 사용
    if (use_xy_model)
        pred = ProjectDLT_XYZtoCR(dltParam2, gcp.xyz, iop); // XYZ → xy → CR
    else
        pred = ProjectDLT_XYZtoCR_Pixel(dltParam1, gcp.xyz); // XYZ → CR 직접

    // 실제 좌표
    double gt_c = gcp.order[0].CR.Col;
    double gt_r = gcp.order[0].CR.Row;

    // 오차 계산
    double dc = pred.x - gt_c;
    double dr = pred.y - gt_r;

    // RMSE 누적
    sum_c += dc * dc;
    sum_r += dr * dr;
    sum_total += dc * dc + dr * dr;

    // 개별 GCP 결과 출력
    cout << left << setw(5) << i + 1
        << setw(10) << static_cast<int>(gt_c)
        << setw(12) << fixed << setprecision(2) << pred.x
        << setw(10) << static_cast<int>(gt_r)
        << setw(12) << pred.y
        << setw(8) << showpos << dc
        << setw(8) << dr << noshowpos << endl;
}

// 최종 RMSE 출력
cout << fixed << setprecision(8);
cout << "Wnc RMSE : " << sqrt(sum_c / N) << " pxWn";
cout << "r RMSE : " << sqrt(sum_r / N) << " pxWn";
cout << "2D RMSE: " << sqrt(sum_total / N) << " pxWn";
};

// 모델점에 대해 두 방식 비교
computeRMSE(model_gcps, "DLT (XYZ→CR) [모델점]", false);
computeRMSE(model_gcps, "DLT (XYZ→xy→CR) [모델점]", true);

// 검사점에 대해 두 방식 비교
computeRMSE(check_gcps, "DLT (XYZ→CR) [검사점]", false);
computeRMSE(check_gcps, "DLT (XYZ→xy→CR) [검사점]", true);
}

#include "Midterm.h"

```

```

#include "FrameCamera.h"
#include "function.h"

double ComputeRMSE(const std::vector<GCP>& gcps, const EOP& eop, const IOP& iop);

void Midterm_3() {

    // 1. 경로 설정
    string folder_dir = "C:/Users/tnddm/OneDrive/바탕 화면/codepaper/opencv/resource/";
    string image_name = "R0047633.JPG";
    string IOP_name = folder_dir + "IOFile_Metric.txt";
    string EOP_name = folder_dir + "input_EOP.txt";
    string GCP_name = folder_dir + "GCP.txt";

    // 2. IOP, EOP, GCP 파일 읽기
    IOP ImageIOP;
    vector<EOP> ImageEOPs;
    EOP ImageEOP;
    vector<GCP> ImageGCPs;
    vector<GCP> ImageGCP;

    ReadIOP(IOP_name, ImageIOP);
    ReadEOP(EOP_name, ImageEOPs);
    ReadGCP(GCP_name, ImageGCPs);

    // 3. 해당 영상의 EOP, GCP 필터링
    ReadImageEOP(ImageEOPs, image_name, ImageEOP);
    ReadImageGCP(ImageGCPs, image_name, ImageGCP);

    // GCP 정보 출력
    PrintGCPInfo(ImageGCP, image_name);

    // GCP CSV 파일로 변경 및 출력
    // SaveGCPToCSV(ImageGCP, image_name);

    // 4. 모델점과 검사점 나누기
    vector<GCP> ModelGCP;
    vector<GCP> CheckGCP;

    //for (int i = 0; i < ImageGCP.size(); ++i) {
    //    if (i < 9) ModelGCP.push_back(ImageGCP[i]); // 앞 5개 모델점
    //    else CheckGCP.push_back(ImageGCP[i]);      // 나머지 검사점
    //}

    vector<int> selectedModelIndices = {1, 2, 5, 6, 8}; // 원하는 GCP 번호들

    for (int i = 0; i < ImageGCP.size(); ++i) {
        // GCP 번호는 1-based로 생각하고 있으므로 (i + 1)
        if (find(selectedModelIndices.begin(), selectedModelIndices.end(), i + 1) != selectedModelIndices.end())
        {
            ModelGCP.push_back(ImageGCP[i]); // 선택된 모델점
        }
        else {
            CheckGCP.push_back(ImageGCP[i]); // 나머지 검사점
        }
    }

    // --- 초기 Mapping 에러 출력 ---
    cout << "WnWnWn===== [ Initial Mapping Evaluation ] =====Wn" << endl;

    cout << "----- [Control Points Mapping Error] -----Wn" << endl;
}

```

```

double RMSE_Model_Before = ComputeRMSE(ModelGCP, ImageEOP, ImageIOP);

cout << "Wn----- [Check Points Mapping Error] -----Wn" << endl;
double RMSE_Check_Before = ComputeRMSE(CheckGCP, ImageEOP, ImageIOP);

cout << "Wn----- [ Mapping RMSE ] -----Wn" << endl;
cout << fixed << setprecision(2);
cout << "Control Points RMSE (Before) : " << RMSE_Model_Before << " px" << endl;
cout << "Check Points RMSE (Before) : " << RMSE_Check_Before << " px" << endl;


cout << "Wn===== " << endl;
cout << "WnWnWn===== [ Precision Sensor Modeling ] =====Wn" << endl;

// --- 센서모델링 (정밀 보정) 수행 ---
EOP CorrectedEOP = PrecisionSensorModel(ModelGCP, ImageEOP, ImageIOP);
// --- 보정 후 Mapping 에러 출력 ---

cout << "----- [Control Points Mapping Error] -----Wn" << endl;
double RMSE_Model_After = ComputeRMSE(ModelGCP, CorrectedEOP, ImageIOP);

cout << "Wn----- [Check Points Mapping Error] -----Wn" << endl;
double RMSE_Check_After = ComputeRMSE(CheckGCP, CorrectedEOP, ImageIOP);

cout << "Wn----- [ Mapping RMSE ] -----Wn" << endl;
cout << "Control Points RMSE (After) : " << RMSE_Model_After << " px" << endl;
cout << "Check Points RMSE (After) : " << RMSE_Check_After << " px" << endl;

cout << "Wn===== " << endl;
}

double ComputeRMSE(const std::vector<GCP>& gcps, const EOP& eop, const IOP& iop) {
    double sumSquaredError = 0.0; // 제공할 누적값
    double sumError = 0.0; // 오차 누적값
    std::vector<double> errors; // 개별 GCP 오차 저장용

    int count = 0;

    // EOP에서 회전행렬 계산 (0, P, K는 degree → radian 변환 필요)
    cv::Mat_<double> R = GetRotationMatrix(eop.O * ToRad, eop.P * ToRad, eop.K * ToRad);

    for (const auto& gcp : gcps) {
        // GCP의 3차원 위치
        double X = gcp.xyz.X;
        double Y = gcp.xyz.Y;
        double Z = gcp.xyz.Z;

        // 실제 영상좌표 (Ground Truth)
        double c_true = gcp.order[0].CR.Col;
        double r_true = gcp.order[0].CR.Row;

        double c_calc, r_calc;

        // Inverse collinearity 식으로 예측된 영상좌표 계산
        InverseMapping(iop, eop, R, X, Y, Z, c_calc, r_calc);

        // 예측좌표와 실제좌표 간의 제공오차 및 오차
        double squaredError = std::pow(c_calc - c_true, 2) + std::pow(r_calc - r_true, 2);
        double error = std::sqrt(squaredError);

        sumSquaredError += squaredError;
        sumError += error;
        errors.push_back(error);
    }
}

```

```

// 개별 GCP 오차 출력
cout << fixed << setprecision(2);
cout << count + 1 << "번 GCP Error : " << error << " px" << endl;

count++;
}

// 평균 오차 및 RMSE 계산
double averageError = sumError / count;
double rmse = std::sqrt(sumSquaredError / count);

// 결과 출력
cout << "Average Error      : " << fixed << setprecision(2) << averageError << " px" << endl;

return rmse;
}

```